



Bahir Dar University

**Bahir Dar Institute of Technology
Faculty of Computing**

**Requirement Analysis Document (RAD)
for
Industrial project on SafeCity+**

**Submitted to the faculty of computing in partial fulfilment of the requirements for the degree
of Bachelor of Science in Computer Science**

Group members

Name	IDNumber
1. Hailemeskel Yihun	1506390
2. Hanna Geremew	1506433
3. Semegn Destaw	1507763

Advisor : Ms.Alemwork M.

**2018
Bahir Dar University, Bahir Dar Institute of Technology**

Declaration

The Project is our own and has not been presented for a degree in any other university and all the sources of material used for the project have been duly acknowledged.

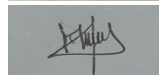
Name

Signature

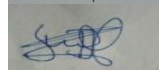
1. Hailemeskel Yihun



2. Hanna Geremew



3. Smegn Destaw



Faculty: Computing

Program: Computer Science

Project Title: SafeCity+

This is to certify that I have read this project and that in my supervision and the students' performance, it is fully adequate, in scope and quality, as a project for the degree of Bachelor of Science.

Alemwork.M



Name of Advisor

Signature

Examining committee members	signature	Date
1. Examiner1		
2. Examiner2		

It is approved that this project has been written in compliance with the formatting rules laid down by the faculty.

Roles and Responsibilities of the Group Members

No	Project Activity / Task	Smegn Destaw	Hanna Geremew	Hailemeskel Yihun
1	Problem Identification & Topic Selection	√	√	√
2	Requirement Gathering & Analysis	√	√	√
3	Literature Review & Background Study		√	√
4	System Analysis & Functional Specification	√	√	
5	Use Case Diagram & UML Design	√		√
6	Database Design	√	√	
7	System Architecture Design	√	√	
8	Frontend Design & UI/UX		√	√
9	Backend Development	√		√
10	Integration & Testing	√	√	√
11	Documentation & Report Writing	√	√	√
12	Presentation Preparation	√	√	√
13	Final Review & Editing	√	√	√

Acknowledgment

We would like to express our sincere gratitude to all those who have contributed to the completion of the documentation phase of this project.

First and foremost, we extend our deepest appreciation to our project guide/supervisor, Alemwork M. for their invaluable guidance, continuous support, and insightful feedback throughout the duration of this project. Their expertise and encouragement have been instrumental in shaping this work.

We are also thankful to Faculty of Computing, Computer Science Department, for providing the necessary resources and a conducive environment to carry out this project.

Our heartfelt thanks to the faculty Computer Science Department for their direct and indirect assistance, which facilitated the smooth progress of our work.

We would like to acknowledge the contributions of our colleagues/classmates, especially, for their collaborative spirit, constructive discussions, and motivation during challenging phases.

A special note of thanks to Amhara Region Police Commission, BahirDar city Administration, MOSEB teams, for their timely and crucial assistance.

List of Acronyms

AES.....	Advanced Encryption Standard
AI.....	Artificial Intelligence
API.....	Application Programming Interface
CCTV.....	Closed-Circuit Television
CNN.....	Convolutional Neural Network
CRC.....	Class Responsibilities Collaborators
ERSS.....	Emergency Response and Support Service
ETA.....	Estimated Time of Arrival
GIS.....	Geographic Information System
GPS.....	Global Positioning System
HTTPS.....	Hypertext Transfer Protocol Secure
ICT.....	Information and Communication Technology
IEEE.....	Institution Of Electronics Engineers
IoT.....	Internet of Thing
KPI.....	Key Performance Indicator
LLM.....	Large Language Model
NFR.....	Non-Functional Requirement
NGO.....	Non-Governmental Organization
NLP.....	Natural Language Processing
OOAD.....	Object-Oriented Analysis Design
OOM.....	Out Of Memory
PMC.....	Programmable Metallization Cell
RAM.....	Random Access Memory
RBAC.....	Role-based Access Control
RON.....	Return On Investment
RUP.....	Rational Unified Process

SDG.....	Sustainable Development Goal
SOS.....	Save Our Souls
SSD.....	Slide State Drive
UI.....	User Interface
UML.....	Unified Modelling Language
Wi-Fi.....	Wireless Fidelity

List of Figures

Figure 1 Use Case diagram	20
Figure 2 State diagram	29
Figure 3 Activity diagram	31
Figure 4 Sequence diagram	33
Figure 5 Class diagram	36
Figure 6 SOS Emergency Reporting	38
Figure 7 SOS Automation pole	39
Figure 8 Emergency Response System	40
Figure 9 Smart city command	41
Figure 10 Prototypes	48
Figure 11 AI-Based Incident Classification	50
Figure 12 Component Diagram	65
Figure 13 Deployment Diagram	68
Figure 14: Class Diagram	73
Figure 15 Persistent Diagram (ERD)	75
Figure 16 App based system design	76
Figure 17 Web based system design	78

List of Tables

Table 1 Functional Requirements	16
Table 2 use Case 1: Register Account	20
Table 3 Use Case 2: Login.....	21
Table 4 Use Case 3: Initiate SOS Alert	22
Table 5 Use Case 4: Submit Emergency Report.....	22
Table 6 Use Case 5: Classify Incident	23
Table 7 Use Case 6: Assign Responder Units	23
Table 8 Use Case 7: Receive Dispatch Alert	24
Table 9 Use Case 8: Close Incident Case	24
Table 10 Business Rules	25
Table 11 Non-Functional Requirement	54
Table 12 Hardware Requirements	55
Table 13 Software Requirements.....	55
Table 14 CRC Analysis	56
Table 15 ChangeCase	58

Abstract

SafeCity+ is an AI-driven smart emergency platform that revolutionizes public safety reporting and response. Through a user-friendly app, citizens can instantly send an SOS alert that automatically shares their GPS location, time, and photo/video evidence. The system's AI analyses this data to classify the emergency type, identify its severity, and importantly assign a priority level to incidents, such as suggesting urgent actions for high-risk accidents and filtering out false alarms.

Emergency responders monitor all incidents through a real-time dashboard featuring interactive maps, multimedia evidence, and AI-generated insights, including priority-based suggestions. This enables faster, coordinated deployment of police, medical, and fire units. By cutting response times, improving decision-making, and enabling predictive safety planning, SafeCity+ helps build proactive, safer, and smarter urban communities.

Table of Contents

Declaration.....	I
Role and Responsibilities of Group Members.....	II
Acknowledgment.....	III
List of Acronyms.....	IV
List of Figure.....	VI
List of Table.....	VII
Abstract	VIII
Chapter One: Introduction	1
1.1Background	1
1.2 Statement of the Problem	1
1.3 Objectives.....	3
1.3.1 General Objective	3
1.3.2 Specific Objectives	3
1.4 Methodology	3
1.4.1 Requirement Gathering Methods.....	4
1.4.2 Analysis and Design Methodology.....	5
1.4.3 Implementation Methodology	5
1.5 Feasibility	6
1.5.1 Economic Feasibility	6
1.5.2 Technical Feasibility.....	7
1.5.3 Time Feasibility.....	7
1.6 Beneficiaries or Significance of the Project	7
1.6.2 Benefits to Emergency Service Providers	8
1.6.3 Benefits to Government and City Administration	8
1.7 Limitations of the Project.....	8
1.7.1 Prototype Implementation	9
1.7.2 Dependence on Internet and GPS Services	9
1.7.3 AI Module Performance Constraints	9
1.7.4 Lack of Direct Integration with Official Emergency Systems	9
1.7.5 Other Operational Limitations	9
1.8 Scope of the Project.....	10
1.8.1 Functional Scope	10

1.8.2 Non-Functional Scope	10
1.8.3 Limitations Within Scope	11
1.9 Organization of the Project	11
Chapter Two: System Features	14
2.1 The Existing System.....	14
2.2 Proposed System	14
2.3 Requirement Analysis	15
2.3.1 Functional Requirements	16
2.3.2 System Use Case	18
2.3.3 Business Rule Documentation.....	25
2.3.4 User Interface Prototype	26
2.3.5 State Chart Diagram	27
2.3.6 Activity Diagram	30
2.3.7 Sequence Diagram	32
2.3.8 Analysis Class Model	34
2.3.9 Logic Model (Algorithms / Pseudocode)	37
2.4 Non-Functional Requirements	54
2.5 System Requirements.....	55
2.6 Key abstraction with CRC analysis.....	56
2.6.1 Identifying Important Concepts.....	56
2.6.2 Identifying Change Cases	57
2.6.3 User Interface Prototyping.....	58
CHAPTER THREE	60
3.1 Architectural Design	60
3.1.1 System Architecture Overview.....	60
3.1.2 Component Modelling.....	61
3.1.3 Deployment Modelling.....	65
3.2 Detailed Design	68
3.2.1 Design Class Model.....	68
3.2.2 Persistent Model	74
3.3 User Interface Design.....	75
3.3.1 Citizen Mobile Application	76
3.3.2 Admin & Responder Web Dashboard	76

3.4 Access Control and Security Design.....	78
Reference.....	81

Chapter One: Introduction

1.1 Background

This project proposes Safe City+, an AI-based smart emergency reporting and response system designed to address the limitations of traditional emergency management in Bahir Dar City. Rapid urbanization has increased the frequency and complexity of emergencies such as traffic accidents, fires, medical crises, and crime, while existing response systems remain largely manual, phone-based, and inefficient. This gap has resulted in delayed responses, inaccurate incident information, and poor coordination among emergency institutions.

Safe City+ provides a centralized, technology-driven solution that enables real-time citizen reporting through mobile and web platforms. The system automatically captures precise geographic locations and allows users to submit multimedia evidence, improving the accuracy and reliability of incident information. Using artificial intelligence, Safe City+ classifies reports, assesses severity, filters duplicates or false submissions, and prioritizes life-threatening emergencies to support faster and more effective response decisions.

The system's micro services-based architecture ensures scalability, flexibility, and long-term sustainability, aligning with Ethiopia's e-Government and National Digital Transformation strategies. A centralized administrative dashboard enhances coordination among police, fire, and medical services, while the citizen-centered design empowers residents to actively participate in public safety.

In conclusion, Safe City+ offers a scalable and policy-aligned approach to modernizing emergency management in Bahir Dar City, with the potential for adoption in other Ethiopian cities to improve response efficiency, institutional coordination, and overall public safety.

1.2 Statement of the Problem

Despite the availability of police, fire brigades, and healthcare facilities, Bahir Dar City's emergency reporting and response system remains inefficient and outdated. Emergencies are mainly reported through phone calls and in-person visits, leading to delayed responses, inaccurate information, and difficulty in identifying the exact location, type, and severity of

incidents. These challenges reduce response effectiveness and increase the risk of loss of life and property.

A major limitation of the current system is the lack of real-time, evidence-based reporting. Emergency operators depend on verbal descriptions from distressed callers, often resulting in missing or incorrect details such as precise location, number of victims, and severity level. This lack of situational awareness causes delays and inefficient allocation of response resources.

Another critical issue is poor coordination among emergency service providers. Police, hospitals, fire brigades, and traffic authorities operate independently using separate communication channels, with no centralized digital platform to support real-time collaboration. This fragmentation slows decision-making and weakens multi-agency emergency responses.

The system also lacks intelligent mechanisms to filter false, duplicate, or low-priority alerts. Reports from the Amhara Regional Police Commission indicate that 25% of emergency calls are false or duplicated (2024). With only 50 police vehicles serving a population of over 500,000, this places significant strain on limited resources and delays responses to genuine emergencies.

Furthermore, the current emergency management approach does not effectively utilize modern digital technologies such as mobile and web-based applications, GIS, real-time location tracking, and Artificial Intelligence (AI). Although national internet penetration is about 25%, smartphone ownership in Bahir Dar has reached 60% (GSMA, 2024), highlighting a missed opportunity to improve emergency response through technology.

These challenges demonstrate the need for a smart, integrated, and intelligent emergency reporting and response system that enables real-time communication, accurate incident classification, automated alert prioritization, and effective coordination among emergency service providers. The proposed SafeCity+ system aligns with Bahir Dar's digital transformation initiatives and aims to enhance public safety and response efficiency.

1.3 Objectives

1.3.1 General Objective

The general objective of this project is to design and develop an AI-based smart emergency reporting and response system for Bahir Dar City.

1.3.2 Specific Objectives

The specific objectives of the SafeCity+ system are to: :

- ❖ Design and implement a web and mobile application that enables citizens to report emergency incidents through a simple, user-friendly SOS action, requiring minimal interaction (ideally one or two taps) for rapid reporting in high-stress situations.
- ❖ Integrate real-time location tracking to automatically capture and transmit the precise geographic location of reported incidents using GPS APIs such as Google Maps or OpenStreetMap.
- ❖ Develop a centralized emergency management dashboard for police, hospitals, and fire brigades to monitor, analyze, and manage emergency alerts in real time.
- ❖ Integrate Artificial Intelligence techniques to classify incident types, prioritize emergencies, and filter false or duplicate reports.
- ❖ Provide real-time status notifications and feedback to users on the progress and handling of their emergency reports.
- ❖ Design the system to be scalable and adaptable for future expansion to other cities in Ethiopia.

1.4 Methodology

This project adopts an object-oriented methodology to analyze, design, and implement the SafeCity+ emergency reporting and response system. This approach ensures a structured process for requirement identification, system modeling, and implementation. By combining object-oriented principles with modern web, mobile, and Artificial Intelligence technologies, the methodology supports scalability, maintainability, and real-time operation in dynamic urban environments. Object-oriented methodology emphasizes concepts such as

encapsulation, inheritance, polymorphism, and abstraction, enabling modular representation of emergency incidents, user interactions, and AI-driven processes.

The methodology is divided into three main phases: requirement gathering, analysis and design, and system implementation. These phases are carried out iteratively with feedback loops to refine results and ensure alignment with stakeholder needs and evolving technologies. This structured yet flexible framework follows software engineering best practices and aligns with insights from McKinsey's Technology Trends Outlook 2025, which highlights rapid advancements in AI and digital infrastructure for scalable applications.

1.4.1 Requirement Gathering Methods

The SafeCity+ system requirements were gathered using multiple complementary methods to ensure accuracy and relevance. These included interviews, field visits, observation, and document analysis, allowing the identification of both functional and non-functional requirements. This approach aligns with Ethiopia's e-Governance Implementation Framework (EeGIF) and considers local socio-economic and technological contexts.

1. Interviews and Field Visits

Interviews and site visits were conducted with key stakeholders from the Amhara Regional Police Commission, Bahir Dar City Administration, and MESOB. These activities helped identify current emergency workflows, coordination challenges, and the need for an integrated digital emergency response system.

2. Observation

Direct observations at police stations, hospitals, and fire departments were used to understand real emergency reporting and response practices, revealing communication delays and coordination gaps.

3. Document Analysis and Literature Review

Relevant government documents, emergency management guidelines, and smart city studies were reviewed to understand existing frameworks and identify best practices and system requirements.

4. Requirement Modelling

All requirements were analyzed and structured using an object-oriented modelling approach to ensure clarity, traceability, and support for future system expansion.

1.4.2 Analysis and Design Methodology

The project adopts an Object-Oriented Analysis and Design (OOAD) methodology to model the SafeCity+ system. This approach supports modularity, reusability, and maintainability by representing real-world entities such as users, incidents, responders, and alerts as system objects. OOAD follows an iterative process inspired by the Rational Unified Process (RUP), making it suitable for AI-integrated systems that require accurate modelling of dynamic behaviors like real-time incident classification.

Unified Modelling Language (UML) diagrams are used to support system analysis and design in accordance with UML 2.5 standards. AI-assisted tools such as Visual Paradigm and Lucidchart were used to generate diagrams efficiently. The main UML diagrams applied include:

- ❖ **Use Case Diagrams** to represent system functions and user interactions.
- ❖ **Class Diagrams** to define system structure, attributes, and relationships, including AI-related components.
- ❖ **Sequence Diagrams** to illustrate interactions between system components over time.
- ❖ **Activity Diagrams** to model workflows and decision logic.
- ❖ **State Chart Diagrams** to describe state transitions of key system components.

These UML models provide a clear visualization of system behavior and interactions, ensuring a shared understanding of the system before implementation.

1.4.3 Implementation Methodology

The SafeCity+ system is implemented using a modular and layered architecture with modern web, mobile, backend, and AI technologies. This approach ensures scalability, flexibility, and ease of maintenance, while Agile-based iterative development supports continuous improvement.

The implementation technology stack includes:

- ❖ **React** for developing the web-based user interface used by administrators and emergency service providers.
- ❖ **React Native** for building the mobile application, enabling citizens to report emergencies quickly via smartphones.
- ❖ **Node.js** with Express for backend development, providing secure and scalable RESTful APIs with JWT-based authentication.
- ❖ **MySQL** for managing structured relational data such as user profiles, incident records, and system configurations.
- ❖ **MongoDB** for storing unstructured and real-time data, including multimedia content and system logs.
- ❖ **Python** for developing the Artificial Intelligence module, supporting incident classification, false alert detection, and decision support.

All system components communicate through secure and well-defined APIs. Testing includes unit, integration, and user acceptance testing, with CI/CD pipelines used to ensure reliable deployment and maintenance.

1.5 Feasibility

The feasibility of the SafeCity+ project was evaluated to determine its practicality, sustainability, and suitability within available resources and institutional constraints. The assessment was conducted from three perspectives: economic, technical, and time feasibility. This evaluation ensures that the system is not only academically sound but also practically implementable within the context of Bahir Dar City, while complying with Ethiopian regulations and existing municipal infrastructures.

1.5.1 Economic Feasibility

The SafeCity+ system is economically feasible as it is developed using open-source technologies such as React, React Native, Node.js, Python, MySQL, and MongoDB, eliminating licensing costs. Development and deployment costs are kept relatively low by leveraging academic resources and free-tier cloud services. The system is expected to reduce operational expenses by digitizing emergency reporting and automating processes, thereby

lowering administrative overheads and manual labor costs. Overall, the benefits of improved efficiency, reduced response delays, and long-term operational savings outweigh the initial development costs, making the project financially viable.

1.5.2 Technical Feasibility

The project is technically feasible since all required hardware, software tools, and platforms are readily available and well supported. The development team possesses adequate skills in web, mobile, backend, database, and AI development to build and integrate the system components. The use of a modular, object-oriented architecture enhances scalability, maintainability, and ease of integration with existing city systems. Common technical risks such as security and system integration are addressed through standard practices including secure APIs, encryption, and role-based access control.

1.5.3 Time Feasibility

The SafeCity+ project is time-feasible and planned to be completed within two academic semesters. The first semester focuses on requirement analysis, system design, and documentation, while the second semester covers implementation, testing, and evaluation. The phased and iterative development approach allows sufficient time for coding, testing, and refinement. Built-in buffer periods and agile practices ensure that potential delays can be managed without compromising project quality or academic requirements.

1.6 Beneficiaries or Significance of the Project

SafeCity+ enhances public safety and emergency response in Bahir Dar City while supporting smart city development. Its scalable and adaptable design allows for future expansion to other cities in Ethiopia, making it a replicable model for technology-driven urban resilience. The project demonstrates how inclusive, AI-enabled solutions can improve quality of life, strengthen institutions, and promote sustainable urban development.

1.6.1 Benefits to Citizens

Citizens are the primary beneficiaries of SafeCity+, as the system enables quick and easy emergency reporting through a simple SOS action. Automatic real-time location tracking

ensures accurate incident identification, reducing delays caused by unclear descriptions. The ability to submit multimedia evidence enhances report credibility and situational awareness for responders. Real-time notifications keep users informed about response progress, reducing anxiety and increasing trust. Overall, the system improves response times, enhances personal safety, and empowers citizens as active participants in their own security.

1.6.2 Benefits to Emergency Service Providers

Emergency service providers, including police, hospitals, and fire brigades, benefit from a centralized dashboard that enables real-time monitoring, incident prioritization, and coordinated response. AI-based incident classification and false alert filtering reduce workload and allow personnel to focus on high-priority emergencies. Improved data sharing and inter-agency coordination lead to better resource utilization, faster decision-making, and enhanced service quality.

1.6.3 Benefits to Government and City Administration

Government and city administrators benefit from data-driven insights generated by the system, including incident trends, response times, and high-risk areas. This supports informed decision-making, strategic planning, and efficient allocation of resources. The system also enhances transparency, accountability, and cost efficiency, contributing to improved urban governance and smart city initiatives.

1.6.4 Benefits to Academic and Research Communities

The SafeCity+ project serves as a practical case study for applying web, mobile, database, and AI technologies to real-world problems. It supports teaching, research, and innovation in software engineering and smart city systems, providing a foundation for future studies, student projects, and interdisciplinary research.

1.7 Limitations of the Project

Despite its potential to improve emergency reporting and response in Bahir Dar City, the SafeCity+ system has certain limitations related to scope, resources, and time. As an academic prototype, the project focuses on demonstrating core concepts rather than

delivering a fully operational city-wide solution. Acknowledging these limitations helps set realistic expectations and highlights areas for future enhancement.

1.7.1 Prototype Implementation

SafeCity+ is developed as a prototype and has not been deployed at full operational scale. While core features are functional and tested in controlled environments, large-scale real-world performance, such as handling high volumes of simultaneous alerts, has not been fully validated. Some system interactions are demonstrated through simulations rather than live integrations, limiting evaluation under real operational conditions.

1.7.2 Dependence on Internet and GPS Services

The system relies on stable internet connectivity and accurate GPS services for real-time reporting and location tracking. In areas with poor network coverage or weak GPS signals, reporting accuracy and response timeliness may be affected. Offline support is limited and cannot fully replace real-time functionalities.

1.7.3 AI Module Performance Constraints

The accuracy of the AI module depends on the availability and quality of training data. Due to limited access to large, diverse local datasets, the AI may produce occasional misclassifications, especially for rare or complex incidents. Continuous data collection and model retraining are required to improve performance over time.

1.7.4 Lack of Direct Integration with Official Emergency Systems

The system is not yet directly integrated with official police, hospital, or fire department systems. This is due to authorization, security, and technical constraints beyond the scope of the academic project. As a result, full automation across existing government workflows is not currently achieved.

1.7.5 Other Operational Limitations

Additional limitations include limited hardware resources, restricted testing across diverse emergency scenarios, and reliance on users' digital literacy. Factors such as older devices,

language barriers, and power or connectivity interruptions may also affect usability and adoption.

1.8 Scope of the Project

The SafeCity+ project focuses on designing and developing an AI-based smart emergency reporting and response system tailored to Bahir Dar City. The system emphasizes real-time incident reporting, alert processing, and responder coordination, leveraging web, mobile, and AI technologies to provide a cohesive, user-centered platform. The scope remains software-focused, avoiding unrelated areas like hardware manufacturing or non-emergency urban services, while allowing future scalability to other cities.

1.8.1 Functional Scope

The system enables citizens to report emergencies quickly via web or mobile applications through an intuitive SOS action. Key functionalities include:

- ❖ Automatic capture of critical data: geographic location, timestamps, and multimedia evidence to support accurate and efficient emergency responses.
- ❖ AI-based analysis: classifies incident types, detects false or duplicate alerts, and prioritizes emergencies.
- ❖ Centralized management dashboard: allows authorized personnel to monitor live alerts, assign responders, update incident statuses, and communicate with both teams and reporting citizens.
- ❖ Analytics and reporting: supports decision-making through insights on response times, incident patterns, and system usage.

This functional scope addresses the core emergency lifecycle from reporting to resolution, emphasizing interoperability, rapid response, and improved situational awareness.

1.8.2 Non-Functional Scope

The project prioritizes software quality and operational performance over physical infrastructure. Key non-functional aspects include:

- ❖ Real-time notifications: push alerts or SMS updates on incident status.

- ❖ **Reliability and availability:** redundancy and error-handling ensure high uptime.
- ❖ **Security:** encryption, role-based access, and compliance with data protection standards.
- ❖ **Scalability:** modular design allows expansion to other cities or additional services.

These measures ensure a reliable, secure, and maintainable system suitable for Bahir Dar's urban conditions.

1.8.3 Limitations Within Scope

To maintain focus and feasibility:

- ❖ **Integration with government systems:** Full linkage to national emergency infrastructures is deferred due to complexity and approval requirements.
- ❖ **Hardware components:** Devices like IoT sensors or smart SOS poles are excluded in this phase, keeping the emphasis on software.

By clearly defining functional and non-functional boundaries, SafeCity+ delivers a robust prototype that enhances emergency response efficiency while leaving room for future expansion and integration.

1.9 Organization of the Project

The SafeCity+ project is structured into chapters and sections to ensure logical flow, clarity, and accessibility for academics, practitioners, and stakeholders. Each chapter is modular yet interconnected, facilitating easy navigation, iterative reading, and comprehension of both conceptual and technical aspects.

Chapter 1: Introduction

Covers the project's context, motivation, and rationale, including:

- ❖ **Background:** Overview of emergency management in Bahir Dar City and urban challenges.
- ❖ **Problem Statement:** Gaps in existing reporting systems, delays, and coordination issues.
- ❖ **Objectives:** General and specific goals aligned with societal and technological benefits.
- ❖ **Methodology:** Requirement gathering, analysis, and implementation strategies.

- ❖ **Feasibility:** Economic, technical, and time assessments to ensure viability.
- ❖ **Beneficiaries & Significance:** Key stakeholders and societal impact.
- ❖ **Limitations:** Scope and resource constraints, guiding future enhancements.
- ❖ **Scope:** Functional and non-functional boundaries of the system.
- ❖ **Project Organization:** Summary of structure and flow for reader orientation.

Chapter 2: System Analysis and Modelling

Focuses on the technical design and system planning, including:

- ❖ **Existing System Analysis:** Identifies gaps in current emergency workflows.
- ❖ **Proposed System Overview:** Objectives, functionalities, and improvements.
- ❖ **Functional & Non-Functional Requirements:** Detailed capabilities and performance expectations.
- ❖ **System Modelling:** UML diagrams (use case, class, sequence, activity) to represent behavior and interactions.
- ❖ **User Interface Prototypes:** Web and mobile design wireframes for diverse stakeholders.
- ❖ **System Architecture & Design Models:** Layered blueprint covering frontend, backend, data storage, scalability, and modularity.

Chapter 3: Architectural & Detailed Design

Covers system construction, user interface, access control, and security, including:

- ❖ **System Purpose & Scope:** Real-time AI-enhanced reporting for citizens, police, and administrators.
- ❖ **Architecture:** Multi-tier client-server design for modularity, scalability, and maintainability.
- ❖ **Deployment & Infrastructure:**
- ❖ **Client devices:** Smartphones and web browsers.
- ❖ **Application server:** Node.js backend with APIs, notifications, AI/GIS integration.
- ❖ **Database servers:** MySQL (structured data) and MongoDB (multimedia/analytics).
- ❖ **Cloud services:** AI processing and GIS visualization for scalable operations.

- ❖ **Design Models:** Core classes (User, Citizen, PoliceOfficer, Admin, CrimeReport, MediaFile, Notification) and hybrid persistent model combining relational and NoSQL storage.
- ❖ **Security & Access Control:** JWT-based authentication, Role-Based Access Control (RBAC), HTTPS, database restrictions, input validation, and audit logging to ensure confidentiality, integrity, and availability.

Chapter Two: System Features

2.1 The Existing System

The emergency reporting and response system in Bahir Dar City is predominantly manual, relying on phone calls or physical visits to police stations, hospitals, or fire departments. This process depends heavily on verbal communication and human intervention, often causing delays and inaccuracies, particularly during stressful situations. Key information such as exact locations, number of casualties, or incident severity may be incomplete or miscommunicated, leading to misdirected responses.

Operators manually record incidents using paper logs, spreadsheets, or simple digital forms, which are prone to errors from mishearing, vague landmarks, or language differences. The system lacks real-time tracking, GPS integration, and multimedia evidence, forcing responders to rely on callers' accounts, increasing the risk of delayed or inadequate responses. For instance, a fire reported as "small" may actually be large, requiring multiple follow-up calls for reinforcements.

Coordination among police, hospitals, and fire services is limited, with each agency maintaining separate communication channels, slowing multi-agency responses. There are no tools for filtering false or duplicate reports, increasing workload and operational strain on emergency personnel.

Overall, the current manual system results in slower response times, inefficient resource use, and lower service quality. It also hampers long-term improvements such as data-driven planning or training based on incident patterns. These limitations highlight the urgent need for a smart, integrated, and AI-enabled emergency response platform to improve accuracy, coordination, and efficiency, transforming the city's emergency management into a proactive and reliable system.

2.2 Proposed System

The proposed system, SafeCity+, is an AI-based smart emergency reporting and response platform designed to address the limitations of Bahir Dar City's existing manual emergency system. Traditional methods rely on phone calls, physical visits, and verbal communication,

often causing delays, incomplete information, and poor coordination among emergency services. SafeCity+ leverages web and mobile technologies, alongside AI, to enhance efficiency, accuracy, and public safety.

SafeCity+ provides citizens with a simple SOS interface on web and mobile applications, enabling quick emergency reporting with minimal interaction. Upon activation, the system automatically captures critical details: GPS location, date and time, and optional multimedia evidence (photos or videos). Multimedia input improves report reliability and reduces misinterpretation.

All data is securely transmitted to a centralized backend server, where an AI module processes the reports. The AI classifies incidents into categories such as fire, medical, crime, or accident and prioritizes them based on severity. It also filters false alerts and duplicates, ensuring efficient allocation of emergency resources.

Emergency responders access information via a centralized dashboard, displaying active incidents on interactive maps with details including type, location, time, multimedia evidence, and response status. Administrators can assign responders, update incidents, and coordinate multiple agencies in real time, improving situational awareness and response efficiency.

Citizens receive automated notifications about their reports' progress, fostering transparency and trust. Administrators can also generate analytics and reports for resource planning, performance evaluation, and identification of high-risk areas, supporting long-term improvements in urban safety.

SafeCity+ features a modular architecture, allowing future expansion to additional features, responder units, and other cities in Ethiopia. The system integrates advanced technologies with practical emergency management, reducing response times, improving accuracy, and enhancing coordination, contributing to smart city development and improved quality of life.

2.3 Requirement Analysis

2.3.1 Functional Requirements

Functional requirements define the key services and operations that SafeCity+ must perform to meet user needs and achieve its objectives. They describe what the system should do, how users interact with it, and how emergency information flows from citizens to responders and administrators. Clear functional requirements ensure reliability and effectiveness in real-world emergencies.

Each requirement is documented with a unique identifier for easy tracking, implementation, and testing. Core operations include:

- ❖ **User management:** registration, authentication, and role-based access.
- ❖ **Emergency reporting:** SOS activation, automatic location capture, and multimedia submission.
- ❖ **Incident processing:** AI-based classification, duplicate/false alert filtering, and priority assignment.
- ❖ **Notifications:** real-time updates to citizens and responders.
- ❖ **Administrative management:** dashboard access, responder assignment, status tracking, and analytics.

Requirements are prioritized:

- ❖ **High priority:** SOS activation, location tracking, dashboard access, and alert transmission.
- ❖ **Medium/low priority:** historical data, reporting, and configuration options.

Requirements are usually summarized in a table with ID, description, actors, priority, and expected outcome. This section ensures both technical and non-technical readers understand the system's essential functions.

In short, functional requirements define the backbone of SafeCity+, guiding development, reducing ambiguity, and ensuring a reliable and efficient emergency management system.

Table 1 Functional Requirements

Requirement ID	Requirement Description	Priority
FREQ-01	The system shall allow citizens to report emergencies using a	High

	single SOS action via web or mobile application.	
FREQ-02	The system shall automatically capture the user's GPS location when an SOS alert is initiated.	High
FREQ-03	The system shall record the date and time of each emergency report.	High
FREQ-04	The system shall allow users to capture and upload multimedia evidence (photos or videos) during emergency reporting.	High
FREQ-05	The system shall transmit emergency data securely to the backend server in real time.	High
FREQ-06	The system shall classify emergency incidents using an AI module (e.g., accident, fire, medical, crime).	High
FREQ-07	The system shall detect and filter false or duplicate emergency alerts using AI analysis.	Medium
FREQ-08	The system shall display emergency alerts on a centralized dashboard for emergency administrators.	High
FREQ-09	The system shall allow emergency administrators to view incident details, including location, media, and reporter information.	High
FREQ-10	The system shall allow administrators to assign appropriate responder units (police, ambulance, fire brigade).	High
FREQ-11	The system shall allow administrators to update the status of an emergency incident.	High
FREQ-12	The system shall notify users about the status of their emergency reports in real time.	High
FREQ-13	The system shall store emergency reports and response data for future reference and analysis.	Medium
FREQ-14	The system shall allow quick SOS access for users without mandatory login.	High
FREQ-15	The system shall generate basic analytical reports for administrators to review emergency trends.	Low

2.3.2 System Use Case

The system use cases describe how different actors interact with SafeCity+ to enable efficient emergency reporting, monitoring, and response. Main actors: Citizen, Responder (Emergency Admin), Backend Server, AI Module.

- ❖ **Citizen:** Reports emergencies via app or web. Provides location, type, multimedia evidence, and contact info. Receives real-time updates, tracks history, and gives feedback.
- ❖ **Responder (Emergency Admin):** Monitors alerts, prioritizes incidents, assigns responders, updates status, manages resources, and generates performance reports.
- ❖ **Backend Server:** Central hub for data storage, processing, authentication, notifications, and module coordination. Ensures data integrity, load balancing, external integration (maps/GIS), and activity logging.
- ❖ **AI Module:** Analyses reports, classifies incidents, detects false or duplicate alerts, predicts severity, suggests responder assignments, and provides insights to admins. Learns over time to improve accuracy.

2.3.2.1 Use Case Diagram

The use case diagram shows how SafeCity+ actors interact with the system for timely and efficient emergency response. It summarizes functional workflows and responsibilities.

Main Use Cases

- ❖ **Report Emergency:** Citizens submit incident details (type, severity, location, multimedia).
- ❖ **Capture Location & Evidence:** GPS coordinates and media files are attached to reports.
- ❖ **Submit SOS Alert:** Immediate alerts for urgent emergencies, prioritized for fast response.
- ❖ **Analyse Incident (AI):** AI evaluates reports, validates data, and supports decision-making.
- ❖ **Classify Emergency Type:** AI assigns incidents to categories (fire, medical, crime).
- ❖ **Filter False/Duplicate Alerts:** AI removes false or repeated reports.
- ❖ **View Emergency Alerts:** Admins monitor active incidents and prioritize responses.
- ❖ **Assign Responder:** Admins allocate teams based on type, location, and resources.

- ❖ **Update Incident Status:** Admins track progress from report to resolution.
- ❖ **Send Notifications:** Real-time updates to citizens, responders, and stakeholders.
- ❖ **View Reports & Analytics:** Admins review past incidents, performance, and trends.

Relationships

- ❖ Report Emergency → Capture Location & Evidence: Location/media is part of reporting.
- ❖ Submit SOS Alert → Analyse Incident: SOS alerts are automatically analysed.
- ❖ Analyse Incident → Classify Emergency Type: Classification occurs during analysis.
- ❖ Analyse Incident → Filter False/Duplicate Alerts: Optional filtering improves reliability.
- ❖ Update Incident Status → Send Notifications: Status updates trigger automatic alerts.

Use Case ID	UC-01
Use Case Name	Register Account
Primary Actor	Citizen
Secondary Actor	System
Description	Allows a new user to create an account in the SafeCity+ system.
Preconditions	User has installed and opened the application.
Postconditions	User account is created and stored in the database.
Main Flow	1. User opens the app. 2. User selects <i>Register</i>. 3. System displays registration form. 4. User enters name, email, phone, password. 5. System validates input. 6. System creates user account. 7. System displays success message.
Alternate Flow	4a. User enters invalid or incomplete data. 5a. System displays error message and requests correction.

Table 3 Use Case 2: Login

Field	Description
Use Case ID	UC-02
Use Case Name	Login
Primary Actor	Citizen
Secondary Actor	System
Description	Allows a registered user to log into the system.
Preconditions	User is registered.
Postconditions	User is authenticated and redirected to home screen.
Main Flow	1. User selects <i>Login</i>. 2. User enters email and password. 3. System validates credentials. 4. System grants access.

Alternate Flow	3a. Invalid credentials entered. 4a. System displays error message
----------------	--

Table 4 Use Case 3: Initiate SOS Alert

Field	Description
Use Case ID	UC-03
Use Case Name	Initiate SOS Alert
Primary Actor	Citizen
Secondary Actor	System
Description	Allows a user to trigger an emergency alert.
Preconditions	User is logged in and location services are enabled.
Postconditions	Emergency alert is created and sent to backend server.
Main Flow	1. User presses SOS button. 2. System captures current location. 1. 3. System opens emergency form.
Alternate Flow	2a. Location unavailable. 3a. System prompts user to enable GPS.

Table 5 Use Case 4: Submit Emergency Report

Field	Description
Use Case ID	UC-04
Use Case Name	Submit Emergency Report
Primary Actor	Citizen
Secondary Actor	System
Description	Allows user to submit emergency details with media evidence.
Preconditions	User has initiated SOS alert.
Postconditions	Emergency report is stored and forwarded for processing.
Main Flow	1. User enters incident details. 2. User attaches photos/videos/audio. 3. User confirms submission.

	4. System validates data. 2. 5. System sends data to backend.
Alternate Flow	4a. Missing required fields. 5a. System requests correction.

Table 6 Use Case 5: Classify Incident

Field	Description
Use Case ID	UC-05
Use Case Name	Classify Incident
Primary Actor	AI Module
Secondary Actor	System
Description	Automatically classifies the emergency type using AI.
Preconditions	Emergency report with media is received.
Postconditions	Incident type and severity are determined.
Main Flow	1. System forwards media to AI module. 2. AI analyzes content. 3. 3. AI returns classification result.
Alternate Flow	2a. AI fails to classify. 3a. System marks incident for manual review.

Table 7 Use Case 6: Assign Responder Units

Field	Description
Use Case ID	UC-06
Use Case Name	Assign Responder Units
Primary Actor	Admin
Secondary Actor	System
Description	Assigns appropriate responders to an incident.
Preconditions	Incident is classified and visible on dashboard.
Postconditions	Responders are assigned and notified.
Main Flow	1. Admin views emergency report. 2. Admin selects responder units. 3. System records assignment. 4. 4. System sends notifications.

Alternate Flow	2a. No responders available. 3a. System suggests nearby units.
----------------	---

Table 8 Use Case 7: Receive Dispatch Alert

Field	Description
Use Case ID	UC-07
Use Case Name	Receive Dispatch Alert
Primary Actor	Responder
Secondary Actor	System
Description	Notifies responder about assigned incident.
Preconditions	Responder is registered and online.
Postconditions	Responder acknowledges or rejects assignment.
Main Flow	1. System sends dispatch alert. 2. Responder views incident details. 5. 3. Responder accepts assignment.
Alternate Flow	3a. Responder rejects assignment. 4a. System reassigns another unit.

Table 9 Use Case 8: Close Incident Case

Field	Description
Use Case ID	UC-08
Use Case Name	Close Incident Case
Primary Actor	Admin
Secondary Actor	System
Description	Closes an incident after resolution.
Preconditions	Incident marked resolved by responder.
Postconditions	Case is closed and archived.
Main Flow	1. Admin reviews case. 2. Admin confirms resolution. 3. System closes case.
Alternate Flow	2a. Admin rejects closure. 3a. Case remains active.

2.3.3 Business Rule Documentation

Business rules define how SafeCity+ operates, ensuring consistent, reliable, and predictable emergency management. They guide data capture, processing, and transmission, maintaining trust among users and responders.

Key points:

- ❖ **Data handling:** Emergency details (location, time, evidence) must follow predefined standards.
- ❖ **System constraints:** Rules manage report prioritization, simultaneous actions, and system behavior under high load or low connectivity.
- ❖ **Decision-making:** Determine when AI handles incidents automatically and when admin review is required.
- ❖ **Consistency & integrity:** Enforce stability, fairness, and order across all operations.

Summary: Business rules translate policies into system actions, ensuring SafeCity+ delivers dependable, efficient, and controlled emergency response.

Table 10 Business Rules

Rule ID	Rule Name	Description	Reference	Revision History
BR-01	SOS Priority Rule	Every SOS alert submitted by a citizen shall be treated as high priority until verified or classified by the AI module or emergency administrator.	Emergency Policy	Initial Version
BR-02	Location Validation Rule	An emergency report must include GPS location data. If GPS data is unavailable, the system shall request manual location input from the user.	Location Services Policy	Initial Version
BR-03	Evidence Requirement Rule	Multimedia evidence (photo or video) is optional but highly recommended to support accurate incident classification.	Reporting Policy	Initial Version
BR-04	AI Incident Classification Rule	The AI module shall automatically classify the incident type based on uploaded multimedia and metadata.	AI Processing Policy	Initial Version

BR-05	False Alert Detection Rule	The AI module shall flag duplicate or suspicious emergency alerts based on similarity in location, time, and media content.	AI Validation Policy	Initial Version
BR-06	Manual Override Rule	Emergency administrators shall have the authority to override AI decisions when necessary.	Administrative Policy	Initial Version
BR-07	Responder Assignment Rule	Emergency administrators shall assign responder units based on incident type, location, and availability.	Response Management Policy	Initial Version
BR-08	Status Update Rule	Every emergency incident must have a status (Reported, Dispatched, On the Way, Resolved, Closed).	Incident Lifecycle Policy	Initial Version
BR-09	Notification Rule	Users shall receive real-time notifications whenever the status of their emergency report changes.	Communication Policy	Initial Version
BR-10	Data Storage Rule	All emergency reports, evidence, and response actions shall be securely stored for audit and analysis purposes.	Data Management Policy	Initial Version

2.3.4 User Interface Prototype

The SafeCity+ user interface is designed using a prototype-based approach to ensure intuitive, fast, and efficient interactions, suitable for high-pressure emergency situations. The interfaces support both citizens and emergency administrators with distinct roles and workflows.

Citizen Interface:

- ❖ Accessible via mobile app and web, prioritizing simplicity and rapid interaction.
- ❖ Prominent SOS button for immediate emergency reporting.
- ❖ Automatic capture of GPS location, date/time, and optional multimedia evidence (photos/videos).
- ❖ Optional text input for additional details.
- ❖ Status-tracking view shows progress (Reported → Dispatched → Responder on Way → Closed) with real-time notifications.

Administrator Dashboard:

- ❖ Web-based interface for police, hospitals, and fire departments.
- ❖ Real-time list of alerts with details: type, reporter, location (interactive map), time, and multimedia.
- ❖ Detailed incident view including AI classification and confidence levels.
- ❖ Assign responder units, update status, add internal notes; updates notify citizens automatically.

AI-Assisted Alerts:

- ❖ AI classifications and confidence levels are integrated into the dashboard.
- ❖ False or duplicate reports are visually flagged for prioritization.
- ❖ Supports informed decision-making without replacing human judgment.

Prototype Tools:

- ❖ Developed using UI prototyping platforms (e.g., Figma) to simulate interactions.
- ❖ Screens align with use cases, enabling early usability testing and iterative refinement.

Summary: The prototype ensures fast, clear, and user-friendly interactions, supporting efficient emergency reporting and management for both citizens and administrators.

2.3.5 State Chart Diagram

The state chart diagram depicts the lifecycle of an emergency incident in SafeCity+, showing system states and transitions from report initiation to resolution. It ensures predictable, consistent handling of emergencies.

Emergency Incident States:

- ❖ **Open App / Emergency Mode:** User launches the app and enters emergency access mode for rapid reporting.
- ❖ **User Login:** Authentication ensures accountability and traceability.
- ❖ **Tap SOS:** User triggers an emergency alert, initiating data capture.
- ❖ **Capture Location & Media:** GPS coordinates, date/time, and optional photos/videos are automatically collected.

- ❖ **Submit Report:** Data is securely sent to the backend for processing.
- ❖ **Backend Processing & AI Analysis:** AI validates, classifies, detects duplicates, and prioritizes incidents.
- ❖ **Send Alert to Responders:** Relevant units are notified with incident details and AI classification.
- ❖ **Responder Review & Decision:** Human responders review the alert and assess if escalation is needed.
 - ❖ **No Escalation:** Response proceeds to resolution.
 - ❖ **Escalation:** Additional units or specialized teams are dispatched.
- ❖ **Resolve Incident:** Responders manage the situation, mitigate hazards, and provide aid.
- ❖ **Close Report:** Incident is marked “Resolved,” final reports filed, and data archived for analysis.

StateTransitions:

Transitions occur automatically (data receipt, AI analysis) or manually (responder assignment, status updates), ensuring controlled and logical incident progression.

Summary: The state chart provides a clear, structured view of incident handling, supporting reliable emergency management and serving as a guide for system implementation and validation.



Figure 2 State diagram

2.3.6 Activity Diagram

The activity diagram illustrates the workflow of emergency reporting and response in SafeCity+, focusing on the sequence of actions and interaction between the citizen, system, AI module, and emergency administrator. It clarifies coordination, process flow, and efficiency.

Activity Flow:

1. **Citizen Opens App:** Launches SafeCity+; interface loads for immediate reporting.
2. **Tap SOS Button:** Triggers emergency workflow and prioritizes processing.
3. **Capture Data:** System collects GPS location, timestamp, and optional photos/videos automatically.
4. **Send Data to Backend:** Emergency information is transmitted to the backend server for processing.
5. **AI Analysis:** AI module classifies the incident, checks for duplicates, and determines severity.
6. **Admin Dashboard Alert:** Processed incident appears on the administrator dashboard for review.
7. **Incident Review:** Administrator examines details, evidence, and AI classification.
8. **Assign Responders:** Admin allocates appropriate emergency units; action is logged in the system.
9. **User Notification:** System informs the reporting citizen that responders are assigned and action is underway.
10. **Update Status Until Resolution:** Incident status is continuously updated from dispatch to resolution, maintaining transparency.

Decision Points:

- ❖ **False or Duplicate Alerts:** AI or admin can flag incidents to prevent unnecessary responses.
- ❖ **Unavailable Responders:** Admin escalates or reallocates units as needed.

Summary: The activity diagram demonstrates a coordinated sequence of actions, showing how SafeCity+ ensures timely, accurate, and efficient emergency response while keeping both users and administrators informed.

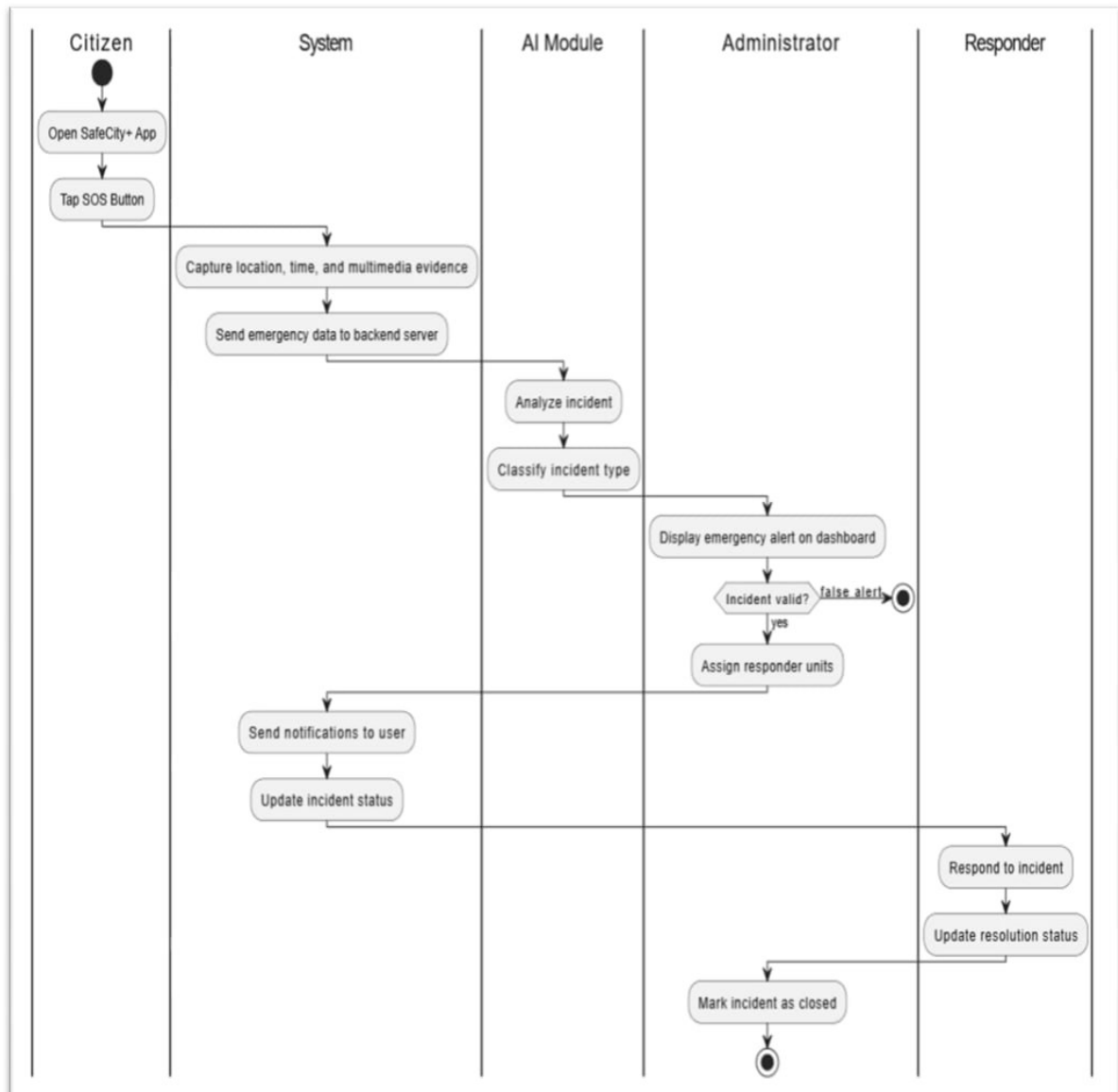


Figure 3 Activity diagram

2.3.7 Sequence Diagram

The sequence diagram shows the time-ordered interactions between SafeCity+ components during emergency reporting, highlighting communication, control flow, and dependencies.

Objects Involved:

- ❖ **Citizen:** Initiates SOS alerts and receives status updates.
- ❖ **Mobile/Web Application:** Captures user actions and sends data to the backend.
- ❖ **Backend Server:** Receives emergency data, coordinates processing, and manages communication.
- ❖ **AI Module:** Analyzes media and metadata, classifies incidents, and returns results.
- ❖ **Emergency Admin Dashboard:** Provides real-time incident access for review, assignment, and status updates.
- ❖ **Notification Service:** Sends real-time updates to citizens.

Interaction Sequence:

1. **Citizen triggers SOS:** Starts emergency processing.
2. **Application sends data:** Collects location, time, and media, then forwards to backend.
3. **Backend forwards media to AI:** AI processes and classifies the incident.
4. **AI returns results:** Classification and indicators sent back to backend.
5. **Dashboard update:** Admins view analyzed incident information.
6. **Responder assignment:** Admin allocates responders; backend records assignment.
7. **Notifications triggered:** Backend notifies the citizen of status changes.
8. **User receives updates:** Citizen gets real-time feedback on incident progress.

Summary: The diagram provides a clear view of component interactions over time, ensuring coordinated, efficient, and reliable emergency response within SafeCity+.

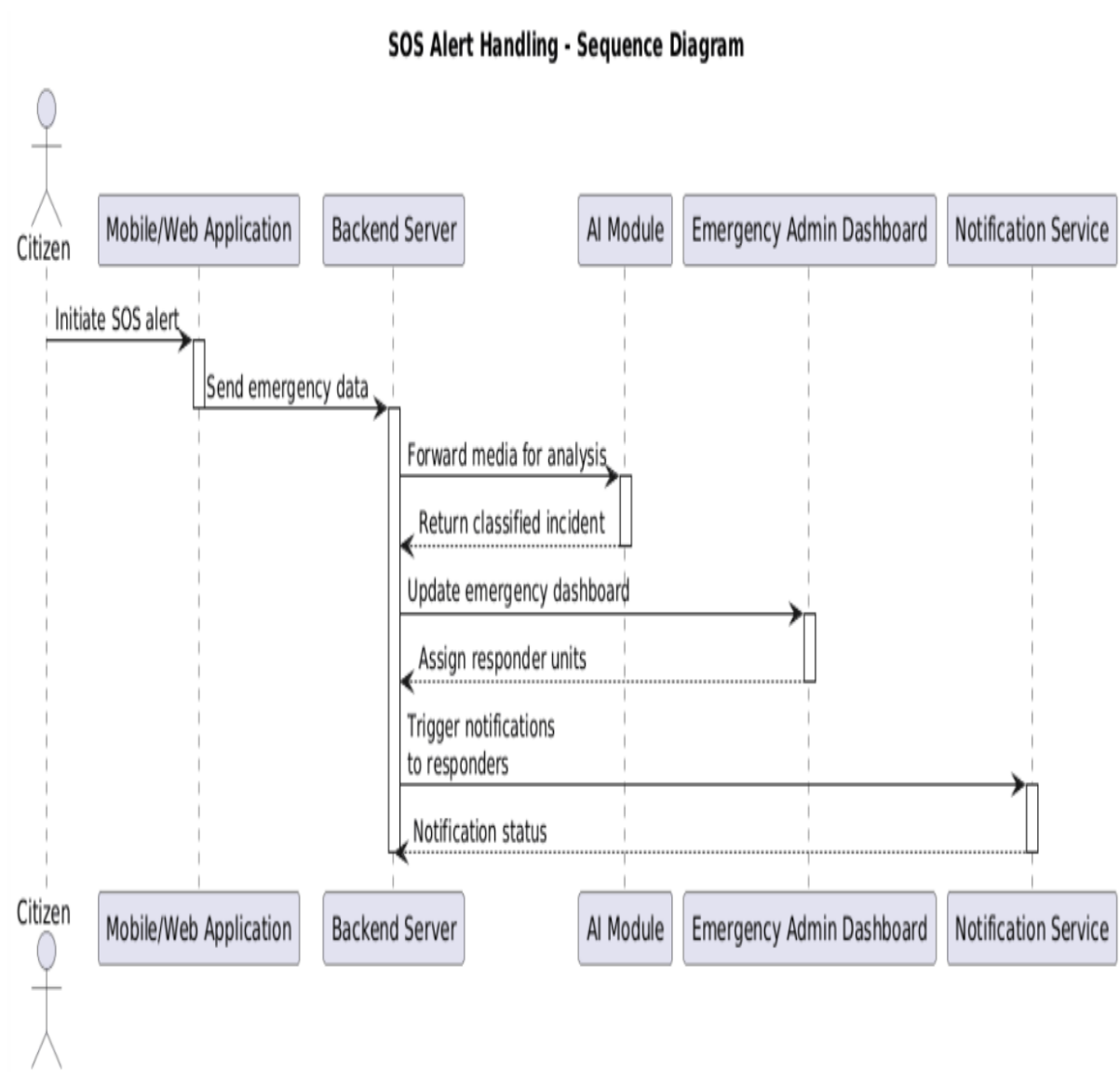


Figure 4 Sequence diagram

2.3.8 Analysis Class Model

The analysis class model provides a conceptual view of SafeCity+, identifying core classes, attributes, methods, and relationships. It focuses on responsibilities and data flow, serving as a foundation for system design.

Main Classes:

User (Citizen): Reports emergencies and receives updates.

- ❖ *Attributes:* userId, name, phoneNumber, deviceId
- ❖ *Methods:* initiateSOS(), captureMedia(), receiveNotification()

EmergencyReport: Represents a reported incident.

- ❖ *Attributes:* reportId, incidentType, location, timestamp, status, mediaEvidence
- ❖ *Methods:* submitReport(), updateStatus(), attachMedia()

EmergencyAdmin: Monitors and manages incidents.

- ❖ *Attributes:* adminId, role, department
- ❖ *Methods:* viewAlerts(), assignResponder(), updateIncidentStatus()

ResponderUnit: Handles incidents in the field.

- ❖ *Attributes:* unitId, unitType, availabilityStatus
- ❖ *Methods:* acceptAssignment(), updateAvailability()

BackendServer: Coordinates and processes system data.

- ❖ *Attributes:* serverId, apiEndpoints
- ❖ *Methods:* processRequest(), storeData(), forwardToAI()

AIModule: Performs automated incident analysis.

- ❖ *Attributes:* modelId, confidenceLevel
- ❖ *Methods:* classifyIncident(), detectFalseAlert()

NotificationService: Sends updates to users.

- ❖ *Attributes:* notificationId, notificationType
- ❖ *Methods:* sendNotification()

Class Relationships:

- ❖ A **User** creates one or more **EmergencyReports**.
- ❖ **BackendServer** processes reports and communicates with the **AIModule**.
- ❖ **EmergencyAdmin** manages reports and assigns **ResponderUnits**.
- ❖ **NotificationService** keeps the **User** informed throughout the incident lifecycle.

SafeCity+ Emergency Response System - Class Diagram

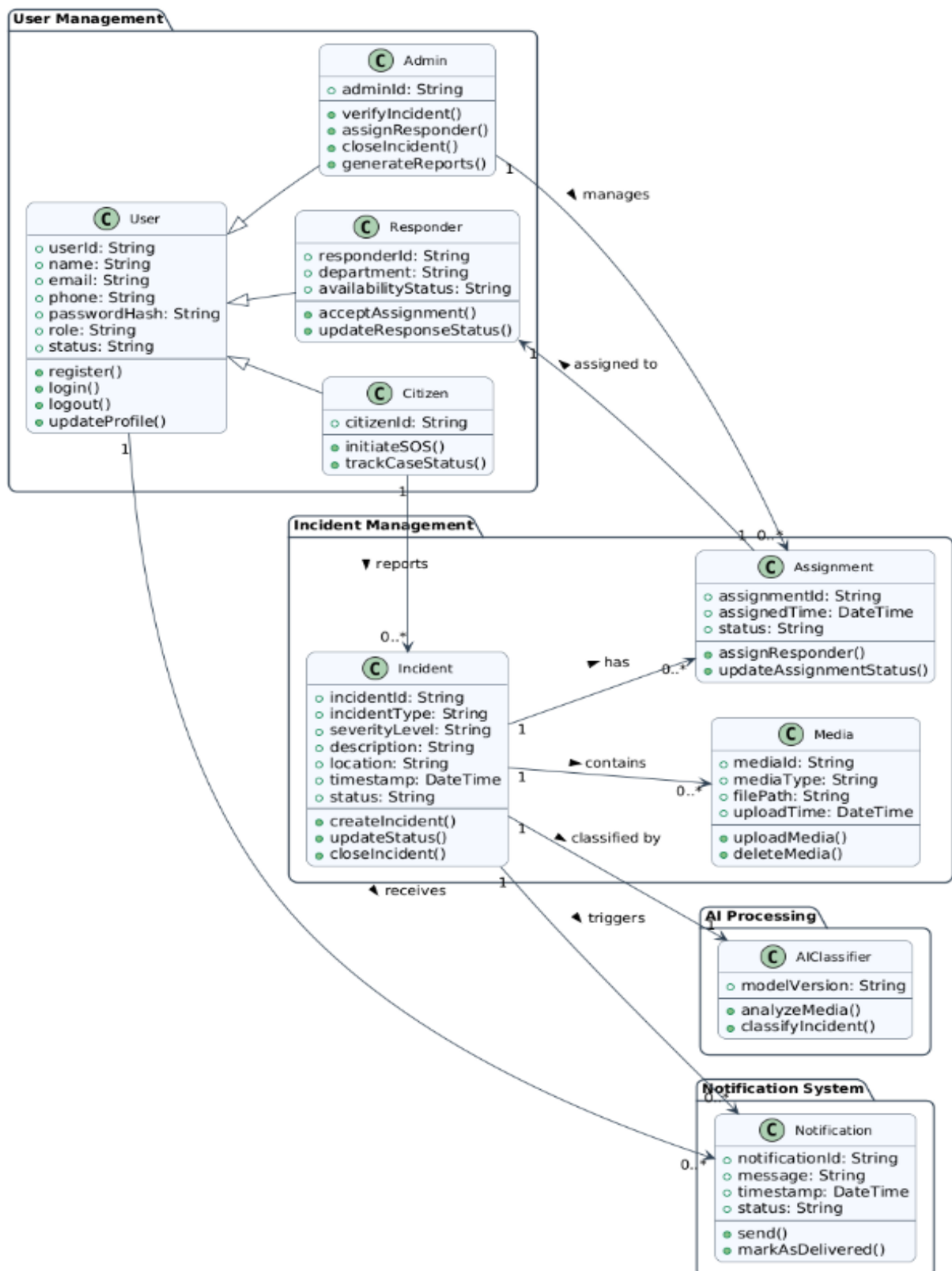


Figure 5 Class diagram

2.3.9 Logic Model (Algorithms / Pseudocode)

This section outlines the logical flow of SafeCity+ SOS emergency reporting, showing step-by-step system behavior when a user triggers an emergency. The logic ensures accurate, reliable, and efficient data capture and transmission.

Algorithm1:SOSEmergencyReporting

When a user taps the SOS button, the system initiates the emergency workflow:

1. Capture the user's GPS location automatically.
2. Record current date and time.
3. Open the device camera for optional media capture.
4. If media is captured, attach it to the report.
5. Compile all collected data into an emergency report.
6. Send the report to the backend server for processing.

Pseudocode:

```
BEGIN
  User taps SOS button
  Initialize emergency reporting
  Capture GPS location
  Capture current date & time
  Open camera for media
  IF media available THEN
    Attach media to report
  ENDIF
  Compile emergency report
  Send report to backend server
END
```

This concise version maintains clarity while highlighting the main logic steps without excessive wording

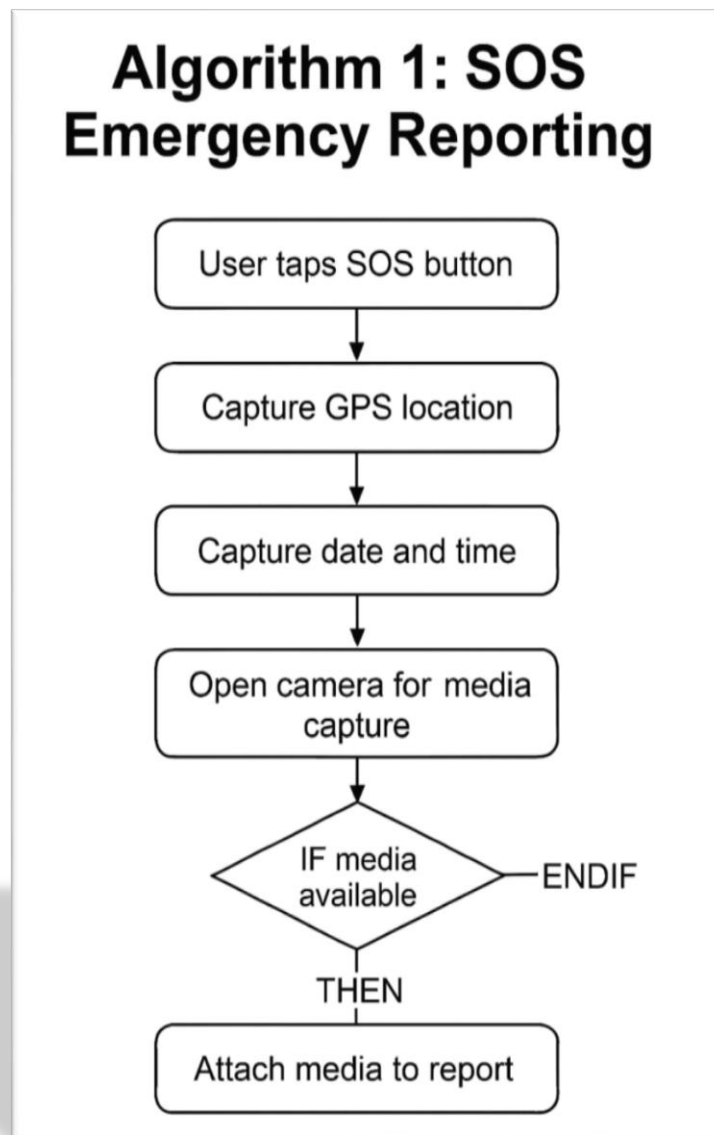


Figure 6 SOS Emergency Reporting



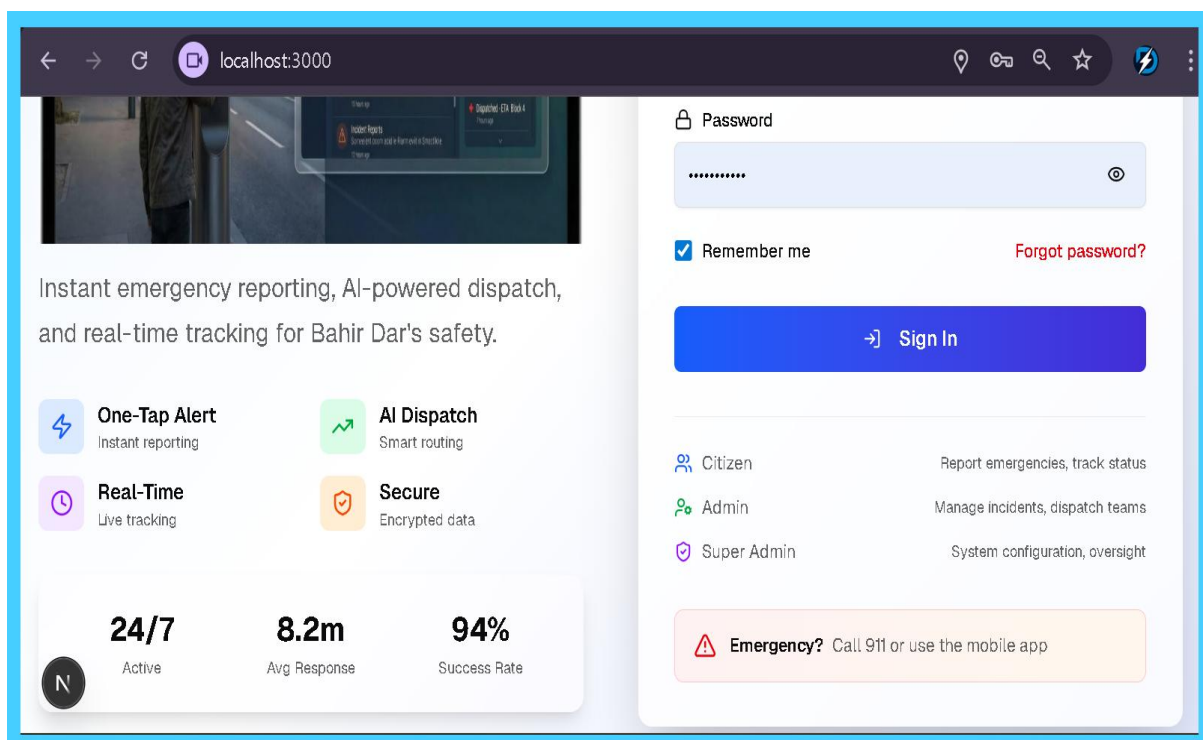
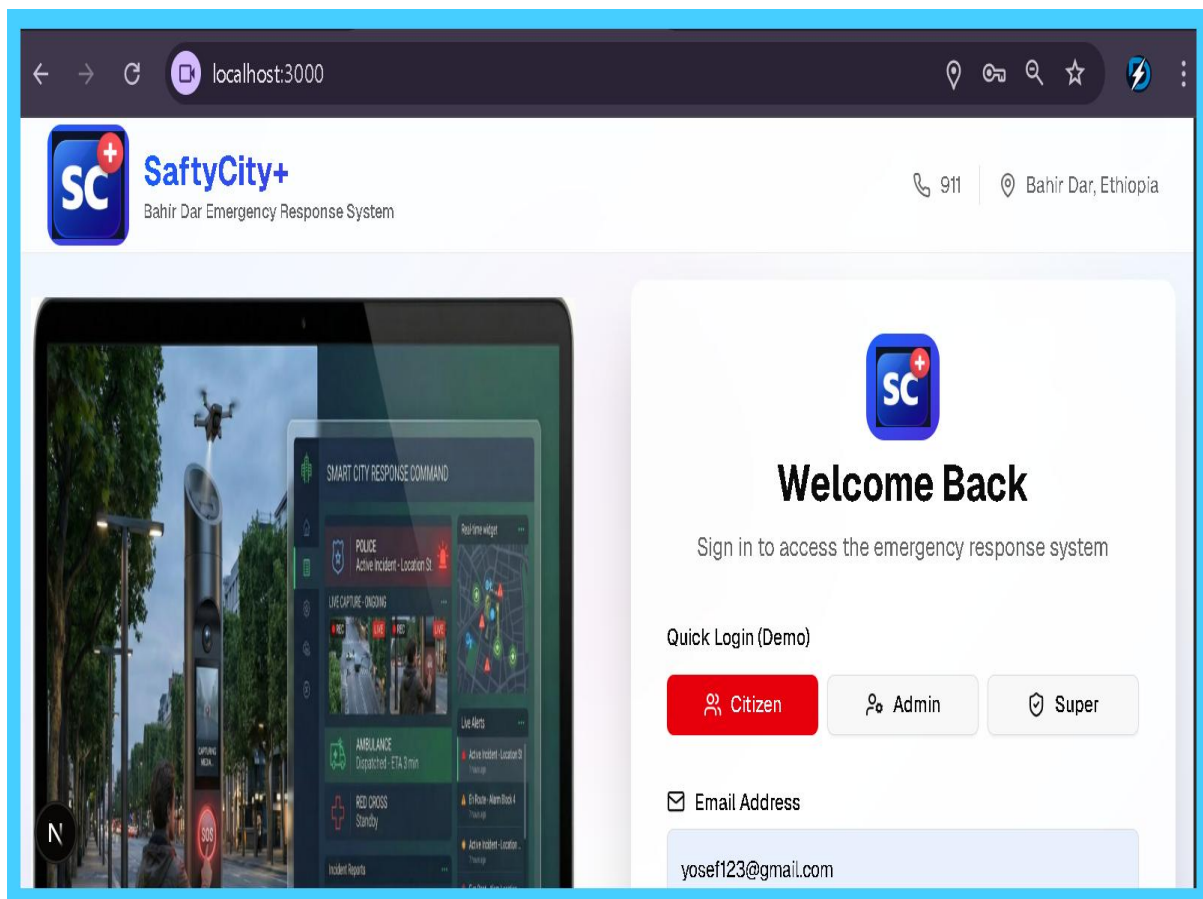
Figure 7 SOS Automation pole

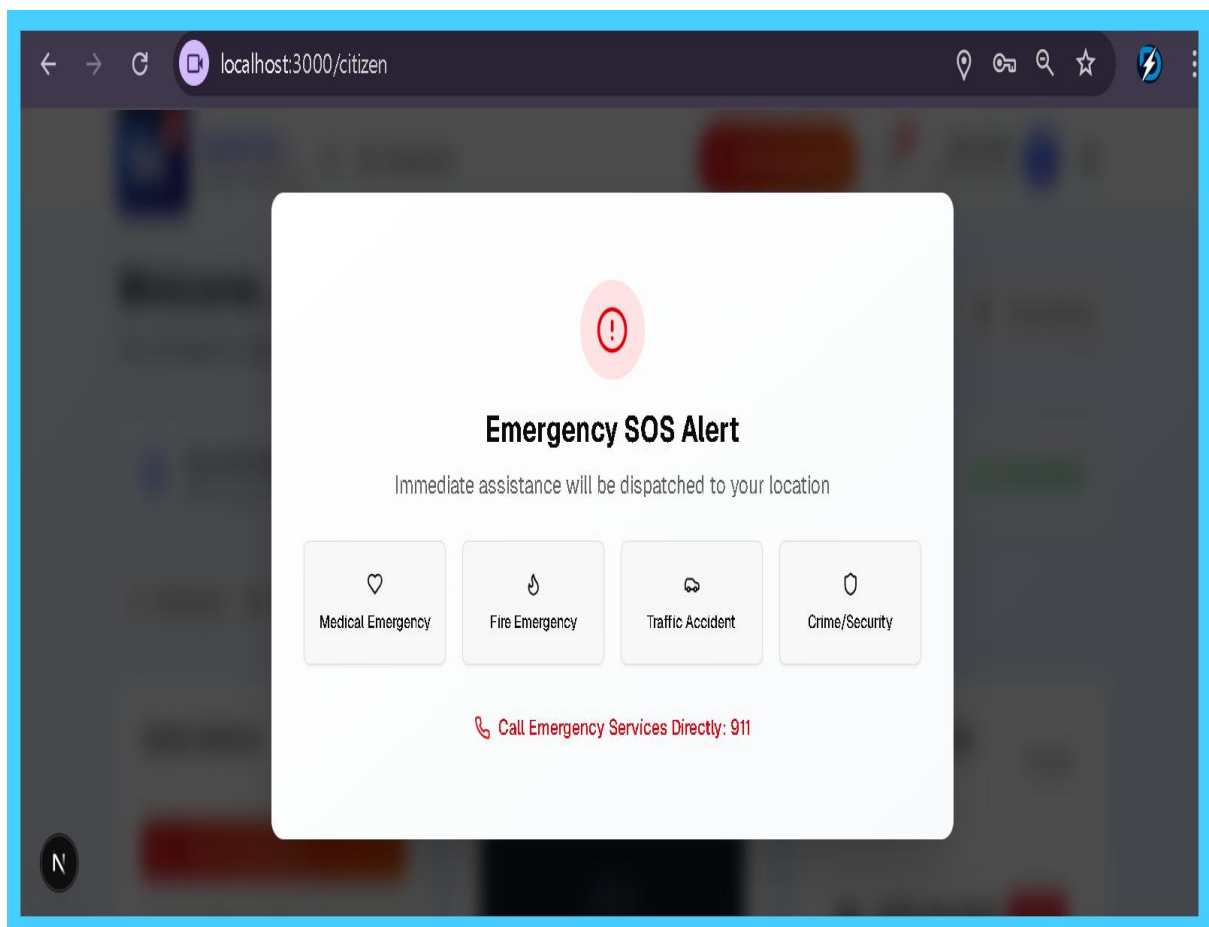
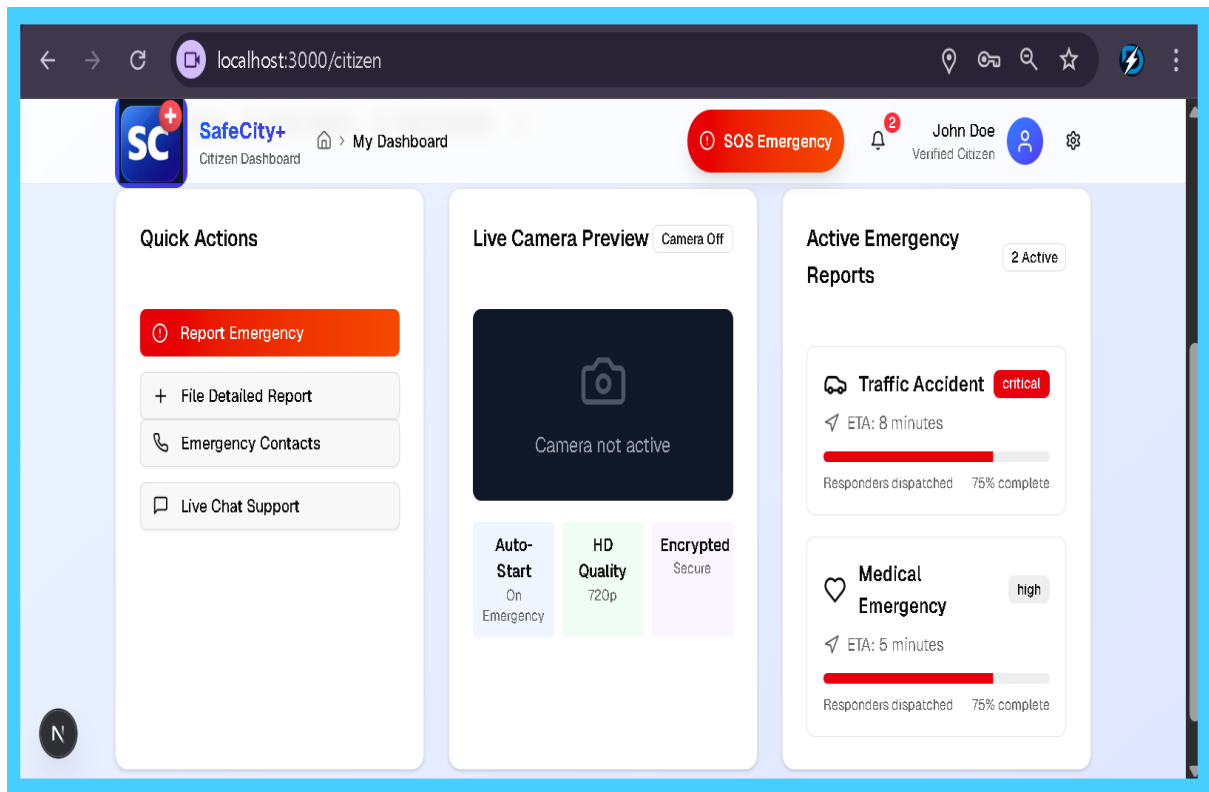


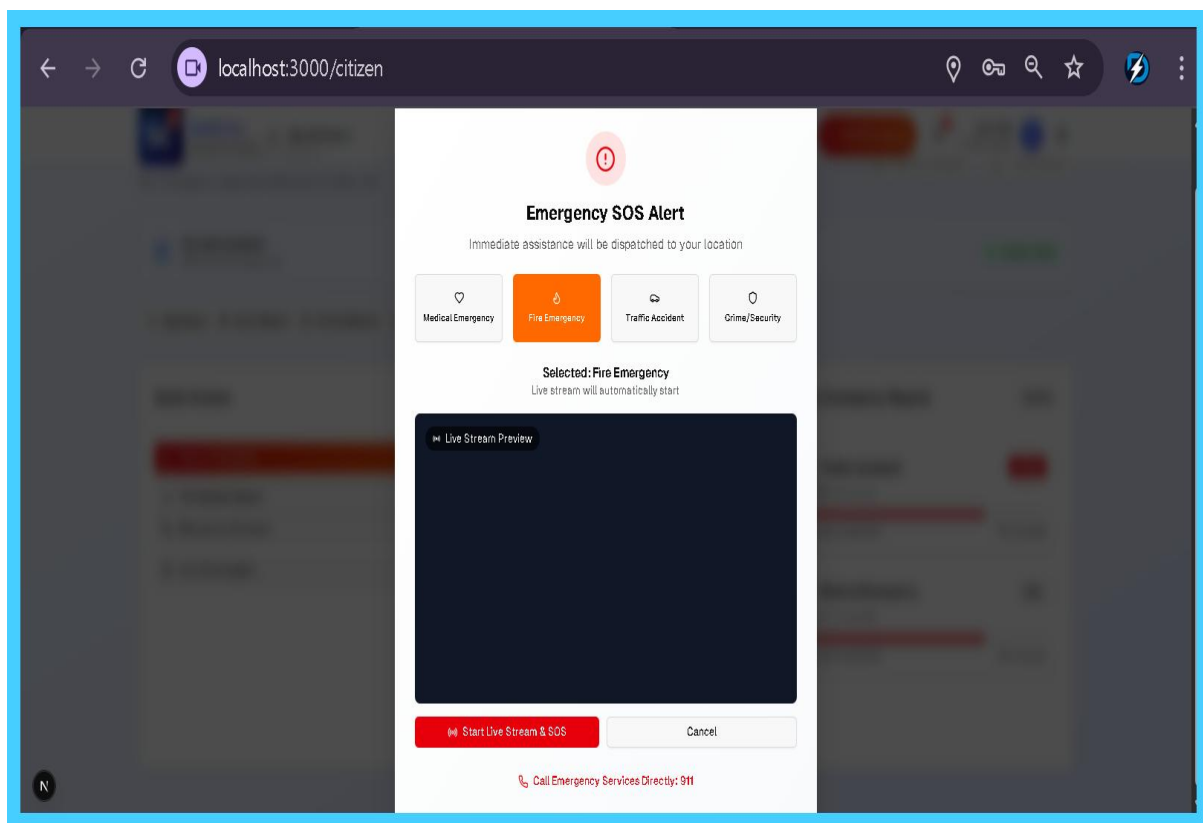
Figure 8 Emergency Response System

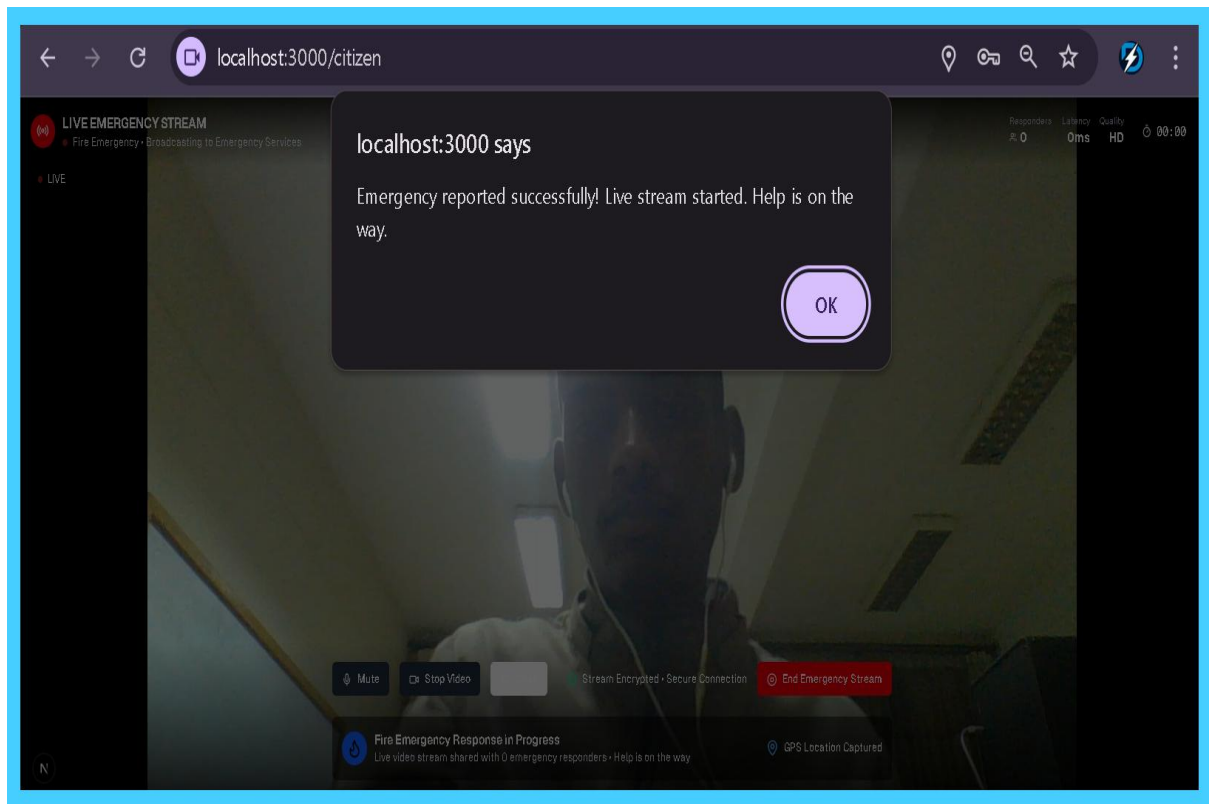


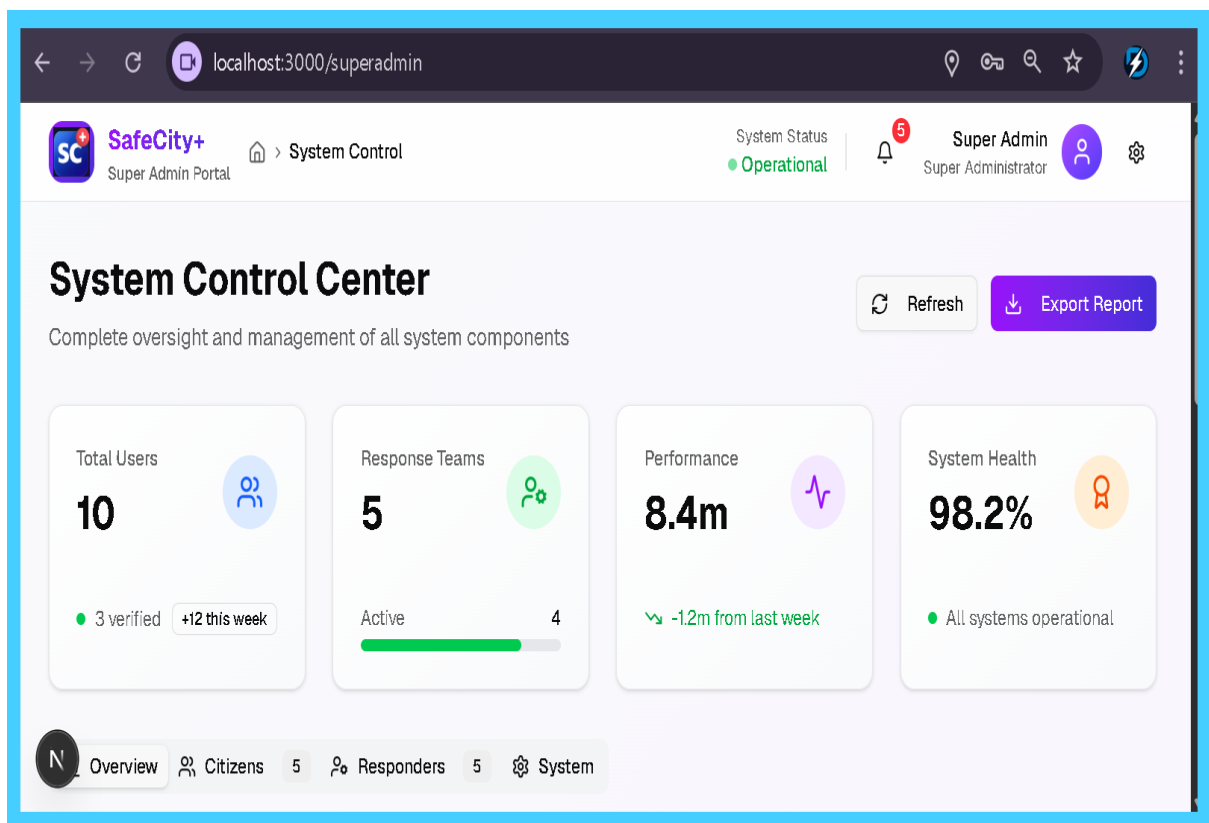
Figure 9 Smart city command

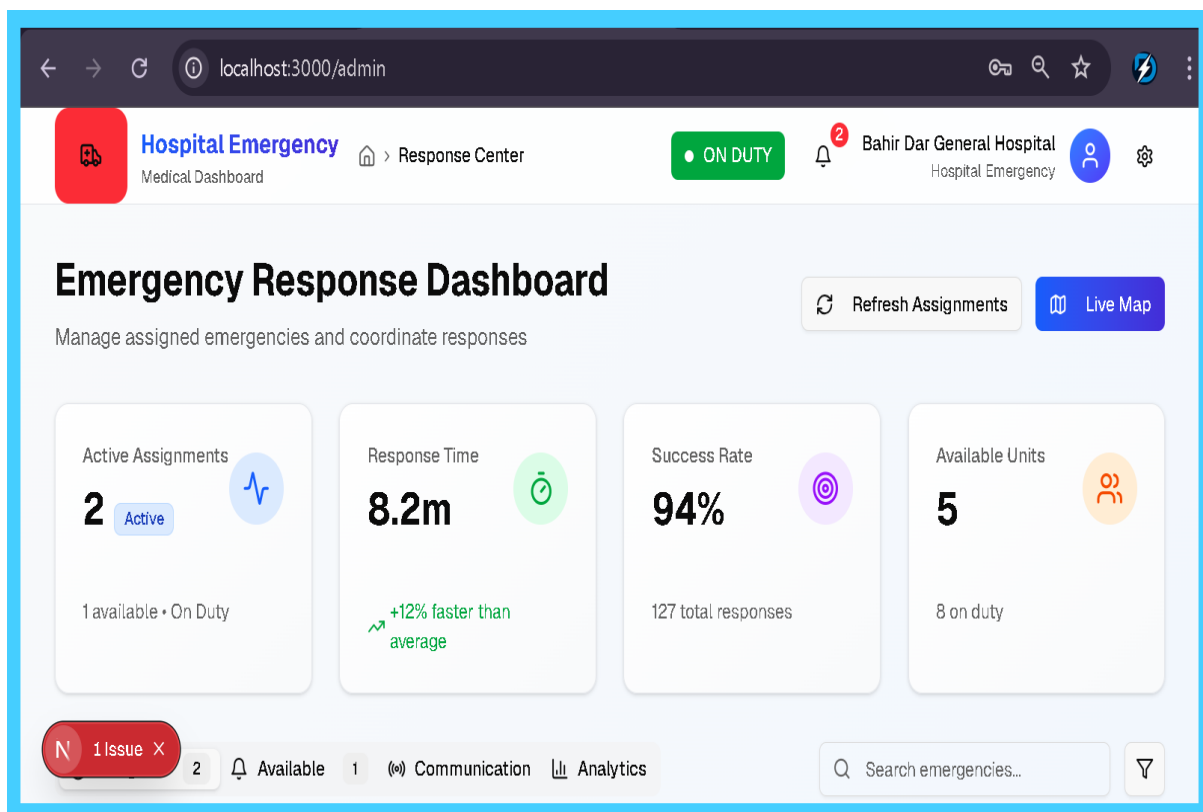
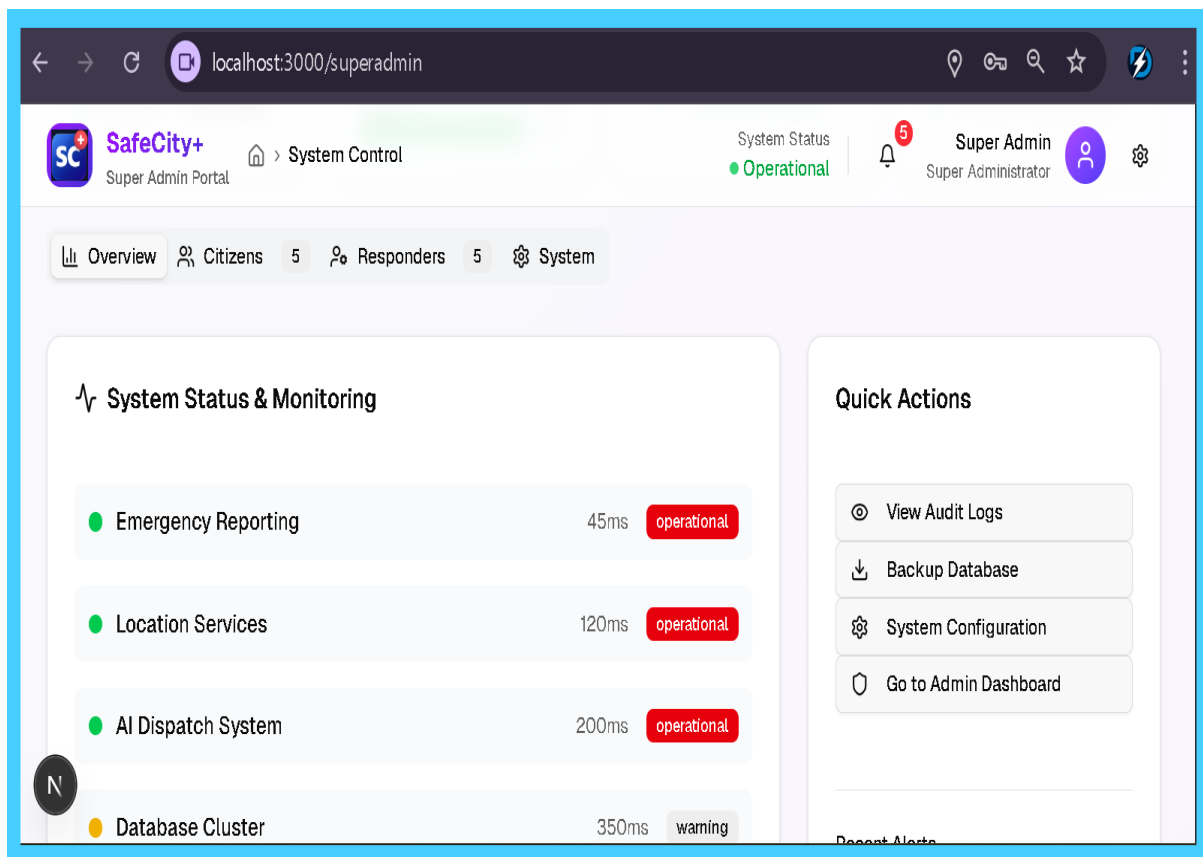












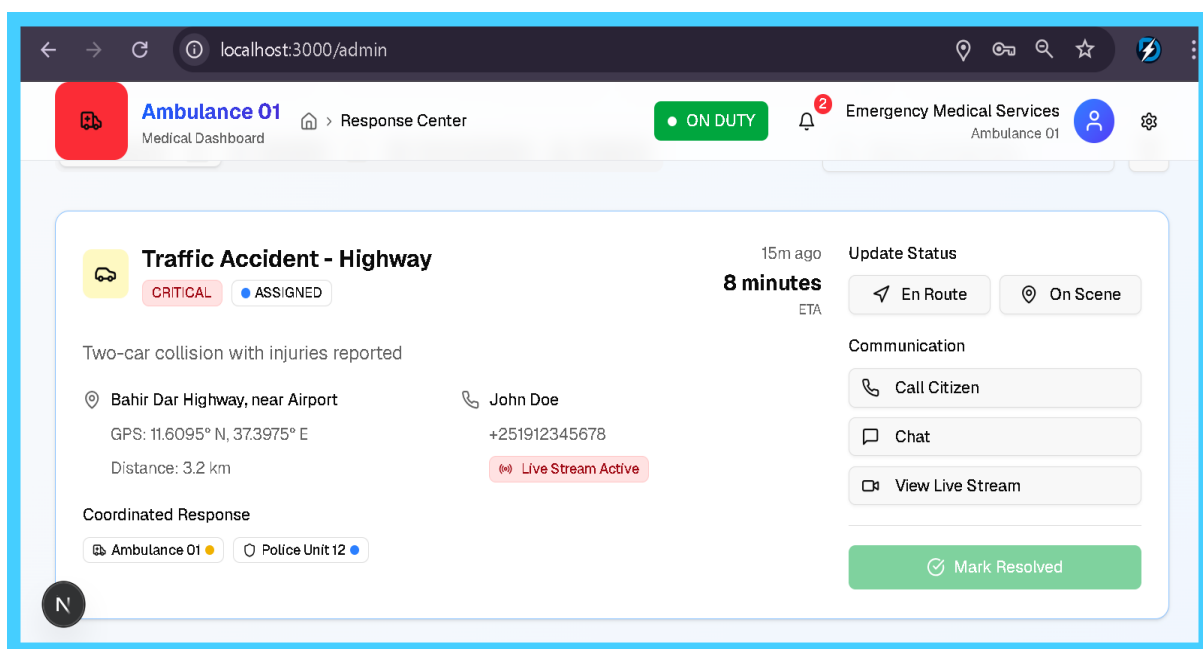
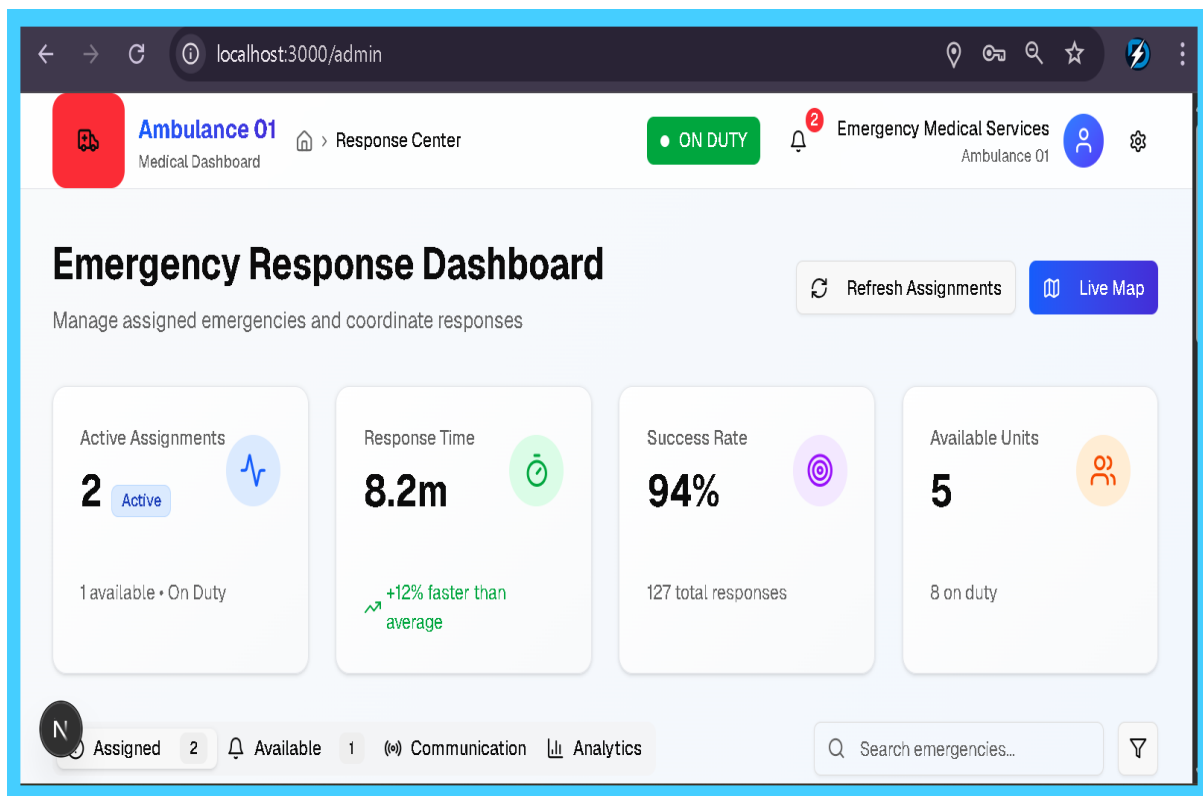


Figure 10 Prototypes

Algorithm 2: AI-Based Incident Classification

This algorithm describes how SafeCity+ uses AI to analyse and classify emergency reports, ensuring accurate and timely response.

Process:

1. Receive emergency report from backend server.
2. Extract multimedia content (images/videos) and metadata (location, timestamp, report ID).
3. Analyze media using AI for patterns and contextual features.
4. Classify the incident into a specific emergency type.
5. Check for duplicates or false alerts.
6. Return classification and validation results to the backend server.

Pseudocode:

```
BEGIN
  Receive report from backend
  Extract media and metadata
  Analyze media with AI
  Classify incident type
  Check for duplicate/false alerts
  Return results to backend
END
```

This version is shorter but still covers AI processing, classification, validation, and feedback.

Algorithm 2: AI-Based Incident Classification

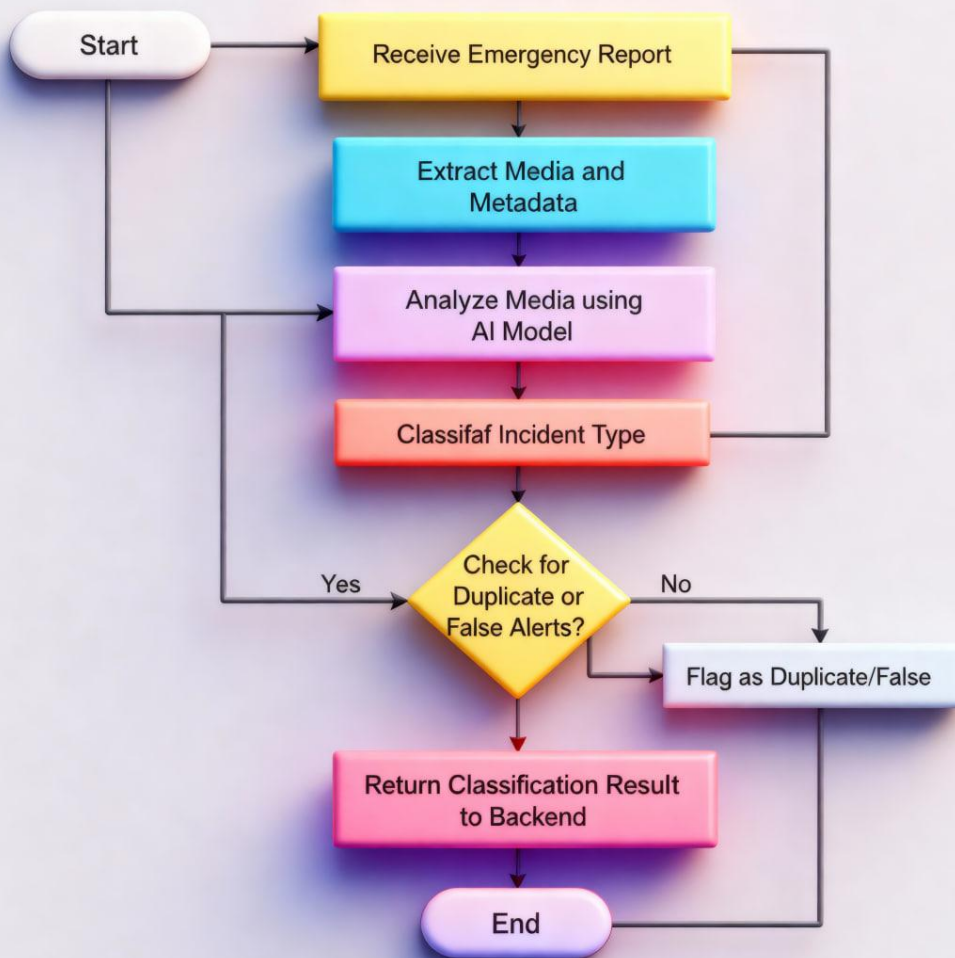


Figure 11 AI-Based Incident Classification

Algorithm 3: Emergency Response Management

This algorithm describes how SafeCity+ manages emergency response, from alert reception to user notification.

Process:

1. Admin receives emergency alert via dashboard.
2. Review incident details (type, location, timestamp, media).

3. Assign appropriate responder unit.
4. Update incident status in the system.
5. Notify reporting user of status updates.

Pseudocode:

```
BEGIN
  Admin receives alert
  Review incident details
  Assign responder unit
  Update incident status
  Notify user
END
```

This concise version emphasizes the **administrative workflow**, responder coordination, and communication without the extra narrative.

Algorithm 3: Emergency Response Management

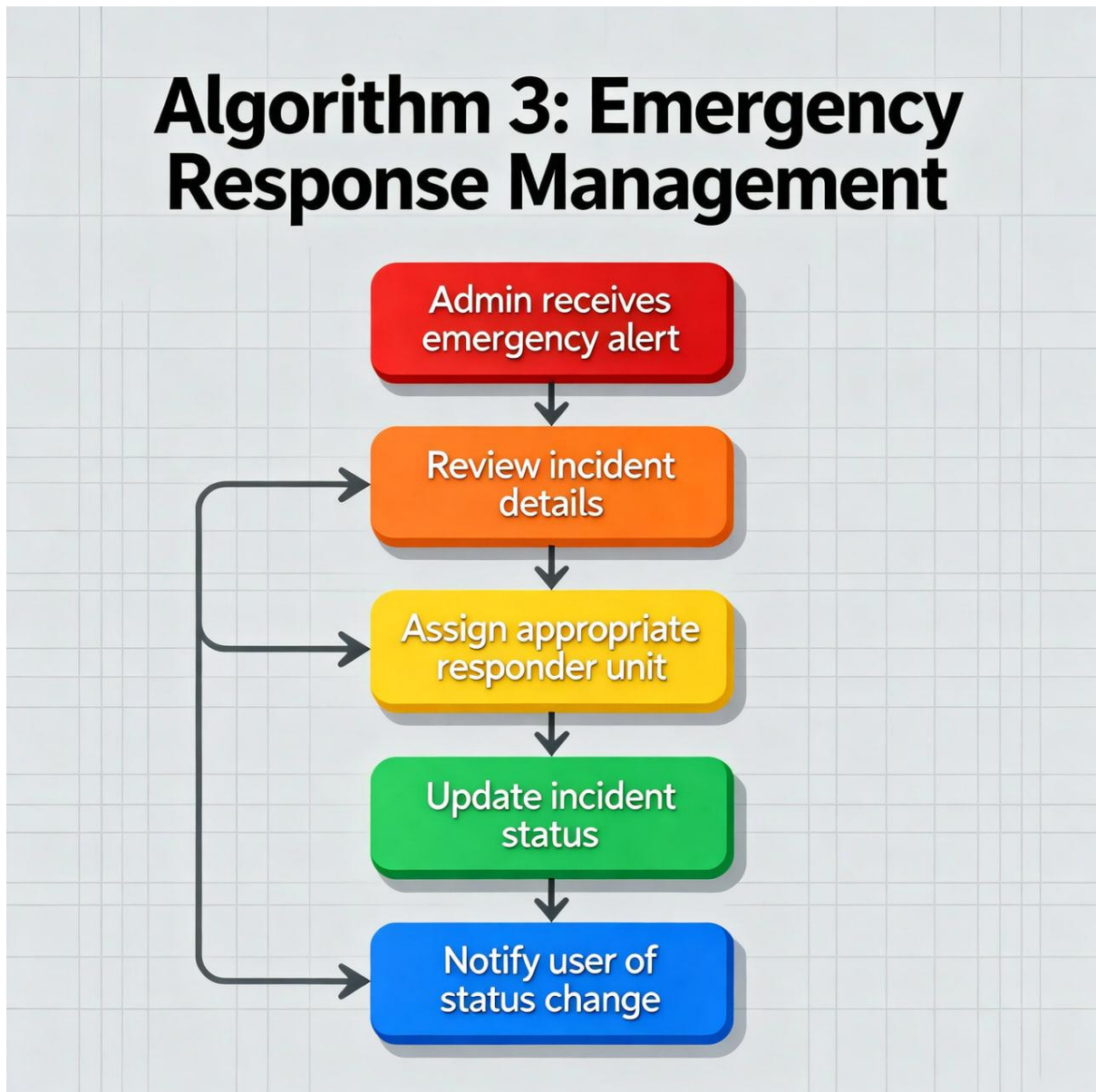


Figure 2.12 Emergency Response Management

Algorithm 4: Notification Process

This algorithm manages real-time updates to users regarding their emergency reports.

Process:

1. Detect status update for an emergency report (e.g., Reported, Dispatched, Responder on the Way, Resolved, Closed).

2. Generate clear, concise notification message.
3. Send notification to user device via push, in-app, or web notification.

Pseudocode:

```
BEGIN
  Detect emergency report status update
  Generate notification message
  Send message to user device
END
```

This version keeps the **core logic** while trimming the narrative.

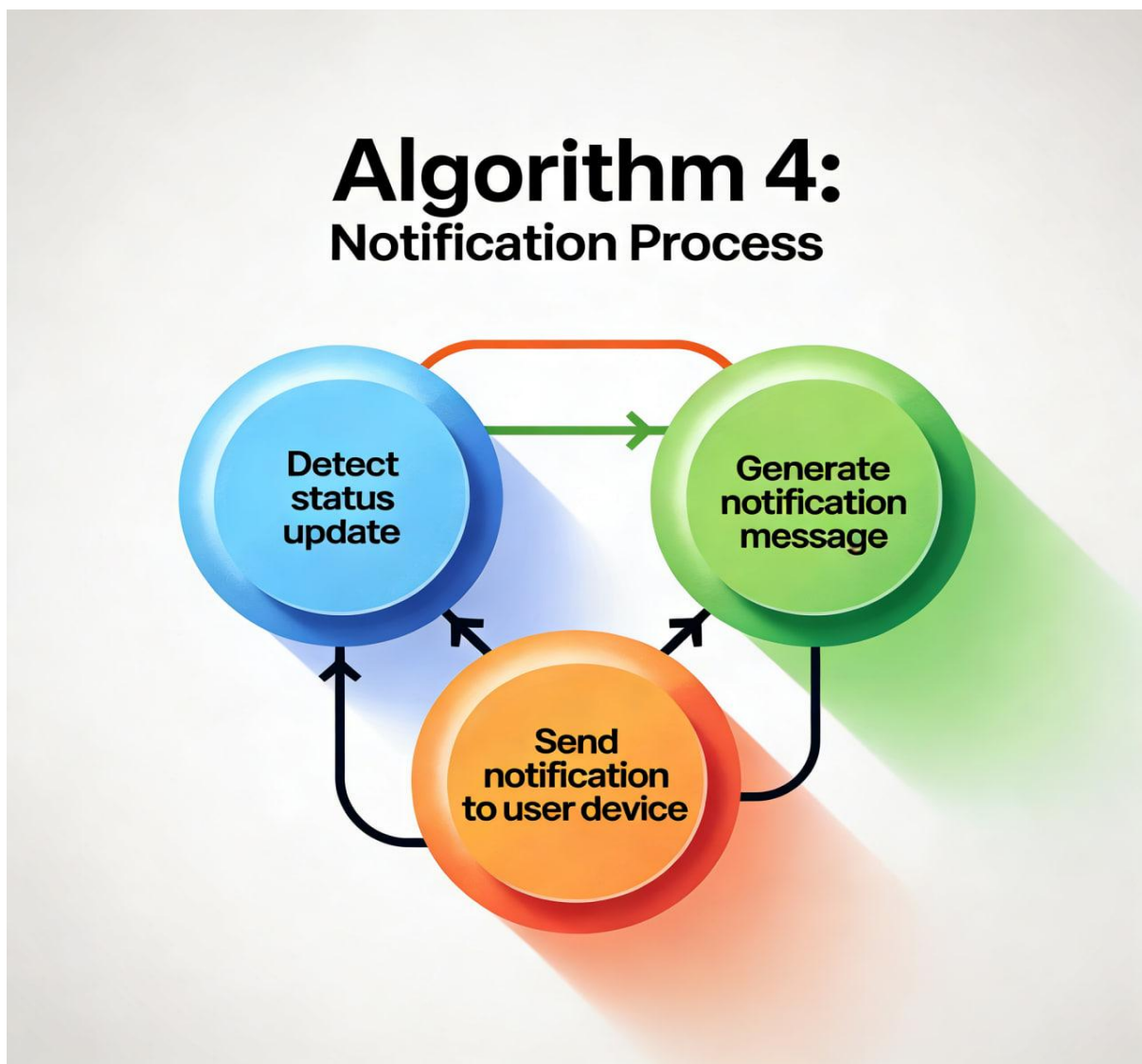


Figure 2.13 Notification Process

2.4 Non-Functional Requirements

Non-functional requirements (NFRs) describe the quality attributes, constraints, and performance expectations of the SafeCity+ system rather than its specific behaviours or functionalities. While functional requirements specify what the system should do, non-functional requirements define how the system performs, how it behaves under various conditions, and the standards it must meet to ensure reliability, security, and efficiency. These requirements are critical because they directly affect user satisfaction, operational effectiveness, and the overall trustworthiness of the system. In the context of SafeCity+, non-functional requirements establish the foundation for a robust, resilient, and high-performing emergency reporting and response platform.

Table 11 Non-Functional Requirement

Category	Requirement	Description
Performance	Response Time	The system shall process and display emergency alerts within 5 seconds to ensure timely response.
Performance	Concurrency	The system shall support up to 500 concurrent users without performance degradation.
Security	Data Privacy	User data, including location and media, shall be encrypted in transit (HTTPS) and at rest.
Security	Authentication	Emergency administrators shall authenticate using secure credentials with role-based access control.
Scalability	System Expansion	The system shall be scalable to include additional cities or regions in the future without redesigning the core architecture.
Reliability	Uptime	The system shall maintain 99% uptime for web and mobile services.
Reliability	Backup	Emergency data shall be backed up daily to prevent loss in case of system failure.
Usability	Accessibility	The user interface shall be simple and intuitive for citizens to quickly report emergencies.
Usability	Multi-platform	The system shall be accessible on modern web browsers and

		mobile devices (iOS & Android).
Maintainability	Modular Design	The system shall follow object-oriented and modular design principles to facilitate future updates and maintenance.
Audit & Reporting	Logging	All system actions, including report submissions, AI classification, and admin updates, shall be logged for auditing purposes.

2.5 System Requirements

System requirements specify the essential hardware, software, and technical resources needed to deploy, operate, and maintain the SafeCity+ system efficiently. These requirements ensure that the system functions optimally, supports all intended features, and provides a seamless experience for both citizen users and emergency administrators. Defining clear system requirements is crucial because inadequate hardware or incompatible software can lead to performance issues, unreliable operation, or security vulnerabilities. By establishing these requirements, the SafeCity+ system can achieve stability, responsiveness, and reliability in real-world deployment scenarios.

Table 12 Hardware Requirements

Component	Specification
Server	Intel i7 or equivalent, 16GB RAM, 500GB SSD, Internet connectivity
User Devices	Smartphones (iOS 13+/Android 10+) or PC with modern browsers
Storage	Database server with at least 1TB storage capacity
Network	Stable internet connection with 10 Mbps minimum for real-time updates
Optional Hardware	Smart SOS poles, CCTV cameras (for future integration)

Table 13 Software Requirements

Component	Specification / Technology
Operating System	Windows Server / Linux (Ubuntu 20.04+)
Web Server	Node.js (latest stable)
Mobile Platform	React Native for iOS & Android
Backend Framework	Node.js with Express
Database	MySQL (relational) + MongoDB (NoSQL for media & logs)

AI & Analytics	Python (TensorFlow / PyTorch for AI classification)
Browser	Chrome, Firefox, Edge (latest versions)
Security	HTTPS, JWT authentication, data encryption (AES)

2.6 Key abstraction with CRC analysis

2.6.1 Identifying Important Concepts

Key concepts the system deals with:

- ❖ User (Citizen)
- ❖ EmergencyReport
- ❖ EmergencyAdmin
- ❖ ResponderUnit
- ❖ AIModule
- ❖ BackendServer
- ❖ NotificationService

Table 14 CRC Analysis

Class	Responsibilities	Collaborators
User (Citizen)	Initiate SOS upload media receive notifications	EmergencyReport BackendServer NotificationService
EmergencyReport	Store incident details manage status attach evidence	User AIModule EmergencyAdmin BackendServer
BackendServer	Receive and Process requests Store data Forward to AI Update dashboard and trigger notification	User EmergencyReport AIModule NotificationService EmergencyAdmin

Class	Responsibilities	Collaborators
AIModule	Classify incident detect false/duplicate alerts Analyse emergency data and media	BackendServer EmergencyReport
EmergencyAdmin	View alerts Assign responders Update status	EmergencyReport ResponderUnit BackendServer
ResponderUnit	Accept emergency assignments Respond to incidents Update availability	EmergencyAdmin EmergencyReport
NotificationService	Generate notification messages Send real-time updates to users	BackendServer User

How This Evolves into a Class Diagram

	Example
Each CRC card → One class	User → initiateSOS()
Responsibilities → Methods	EmergencyReport ↔ AIModule
Collaborators → Associations	EmergencyAdmin ↔ ResponderUnit

:

This CRC analysis directly supports the analysis class model and UML class diagram already described in the SafeCity+ document.

2.6.2 Identifying Change Cases

Change cases describe possible future modifications to system requirements, their likelihood, and their impact on the system. Identifying change cases helps in designing a flexible and adaptable system.

Table 15 ChangeCase

ChangeCase ID	Description of Potential Change	Likelihood	Impact
CC-01	Integration with national police, hospital, and fire department systems	High	High
CC-02	Addition of IoT devices (CCTV, SOS poles, sensors)	Medium	High
CC-03	Support for offline SOS reporting	Medium	Medium
CC-04	Expansion to other cities in Ethiopia	High	Medium
CC-05	Addition of voice-based emergency reporting	Low	Medium
CC-06	Improvement of AI models with real-time learning	Medium	High

High likelihood & high impact changes (e.g., system integration) require modular architecture. Medium likelihood changes should be considered during design.

2.6.3 User Interface Prototyping

User Interface (UI) prototyping is used to visualize system functionality, validate requirements, and gather feedback from users before implementation.

Citizen User Interface Prototype

The citizen interface is designed for quick emergency reporting with minimal interaction. The main screen contains:

- ❖ A large **SOS button**
- ❖ Automatic GPS capture
- ❖ Camera access for photo/video evidence
- ❖ Optional text description field
- ❖ Real-time status display (Reported, Dispatched, On the Way, Resolved)

This design ensures usability during stressful emergency situations.

Emergency Administrator Dashboard Prototype

The administrator interface is a web-based dashboard that provides:

- ❖ Real-time list of emergency alerts
- ❖ Incident details (location map, time, media)
- ❖ AI-classified incident type with confidence level
- ❖ Options to assign responder units
- ❖ Incident status update controls

AI-Assisted View

An AI-assisted panel highlights:

- ❖ Classified incident type
- ❖ Confidence indicators
- ❖ Alerts flagged as false or duplicate

Prototyping Tools

The UI prototypes are created using tools such as:

- ❖ Figma
- ❖ Adobe XD
- ❖ Balsamiq

These prototypes help stakeholders understand system behaviour and enable early requirement validation.

CHAPTER THREE

3.1 Architectural Design

This chapter presents the system design of the **SafeCity+ Crime Reporting and Management System**. The design phase translates the requirements identified in Chapters One and Two into a structured technical solution that defines how system components interact, how data flows, and how users access the system securely.

The SafeCity+ system follows a **client-server architecture** with a modular and scalable design. It integrates web, mobile, backend services, database systems, and AI-assisted analysis to provide efficient crime reporting and management.

3.1.1 System Architecture Overview

The SafeCity+ system is designed using a **multi-tier architecture**, consisting of the following layers:

❖ **Presentation Layer**

- Web application (Admin and Police Officers)
- Mobile application (Citizens)

❖ **Application Layer**

- Backend server and RESTful APIs

❖ **Data Layer**

- Relational and NoSQL databases

❖ **AI & Analytics Layer**

- Crime pattern analysis and prediction

❖ **GIS Integration Layer**

- Location-based crime visualization

This layered architecture improves **maintainability**, **security**, and **scalability**, while allowing independent development of system components.

- ❖ Shows Mobile App, Web App → Backend Server → Databases → AI & GIS Modules

3.1.2 Component Modelling

Component modelling illustrates the major functional modules of the SafeCity+ system and their interactions. The system is composed of several interrelated components, each responsible for a specific set of functions. Together, these components ensure efficient crime reporting, case handling, data analysis, and communication between users and authorities.

Main System Components

1. User Management Component

The User Management Component is responsible for controlling access to the system while balancing security and emergency responsiveness. In addition to standard user registration and authentication, this component supports an Emergency Access Mode that allows urgent reports to be submitted without prior sign-in.

a) User Registration and Authentication

Registered users (Citizens, Police, and Admins) authenticate using secure credentials. Authentication enables full access to system features based on the assigned role, such as report tracking, case management, and administrative control.

b) Role-Based Access Control

The system enforces role-based permissions:

- ❖ **Citizen:** Submit reports, attach evidence, and track case status
- ❖ **Responder:** Review, manage, and resolve cases
- ❖ **Admin:** Manage users, monitor system activity, and configure policies

Each role is restricted to authorized actions to ensure accountability and data protection.

c) Emergency Access Mode (Pre-Login Reporting)

To support urgent situations, the system allows users to submit an emergency report without signing in. This mode is intentionally limited in functionality and is designed only for immediate response scenarios.

Key characteristics of Emergency Access Mode:

- ❖ Accessible before sign-in
- ❖ Allows submission of basic emergency reports only
- ❖ Automatically captures GPS location, timestamp, and device metadata
- ❖ Supports limited media attachments (optional)
- ❖ Does not allow report editing or case tracking without later authentication

Once the emergency is addressed, users may be encouraged to register or sign in to view updates or provide additional details.

This approach ensures that no critical incident is blocked by authentication barriers, while still maintaining system security and preventing misuse through access limitations.

2. Crime Reporting Component

The Crime Reporting Component serves as the primary interface through which citizens report incidents. It allows users to submit detailed crime reports using text descriptions, as well as supporting multimedia evidence such as images and videos.

An important feature of this component is location tagging using GPS. When a report is submitted, the system automatically captures the user's geographic coordinates, ensuring accurate identification of the incident location. This reduces ambiguity and helps authorities respond more effectively.

This component ensures that all submitted reports are validated, timestamped, and securely transmitted to the backend system, forming the foundation for further case processing.

3. Case Management Component

The Case Management Component is used mainly by police officers and administrators to handle reported incidents after submission. Once a crime report is received, it is reviewed and either approved or assigned to the appropriate authority.

This component supports case assignment, enabling supervisors to allocate cases to specific officers or departments based on location, crime type, or workload. It also provides status tracking, allowing cases to move through predefined stages such as:

- ❖ Pending
- ❖ In Progress
- ❖ Resolved

Status updates are recorded in real time, creating transparency and enabling citizens to monitor the progress of their reports. This structured workflow improves accountability and ensures systematic handling of incidents.

4. **Notification Component**

The Notification Component ensures timely communication between the system and its users. It delivers real-time alerts and updates related to report submissions, case status changes, and system announcements.

Notifications are sent through multiple channels, including:

- ❖ **In-app notifications** for immediate visibility
- ❖ **Email notifications** for important updates and confirmations

For example, citizens receive alerts when their report is submitted or when its status changes, while police officers are notified when new cases are assigned. This component enhances responsiveness and keeps all stakeholders informed.

5. **AI Analysis Component**

The AI Analysis Component enhances the system by providing intelligent insights based on collected crime data. It analyses historical and real-time reports to identify crime trends and patterns.

One of its key functions is hotspot prediction, where the system uses data analytics and machine learning techniques to identify areas with a high likelihood of criminal activity. These predictions help law enforcement agencies allocate resources more effectively and adopt preventive measures.

This component transforms raw data into actionable intelligence, supporting data-driven decision-making and proactive crime prevention.

6. Data Management Component

The Data Management Component is responsible for the secure storage, organization, and retrieval of all system data. This includes user profiles, crime reports, multimedia files, case records, and analytical results.

It ensures data integrity, consistency, and availability by using structured databases and access controls. Backup and recovery mechanisms are also implemented to prevent data loss.

By centralizing data management, this component supports efficient system performance, reliable reporting, and long-term analysis.

SafeCity+ Emergency Response System - Component Diagram

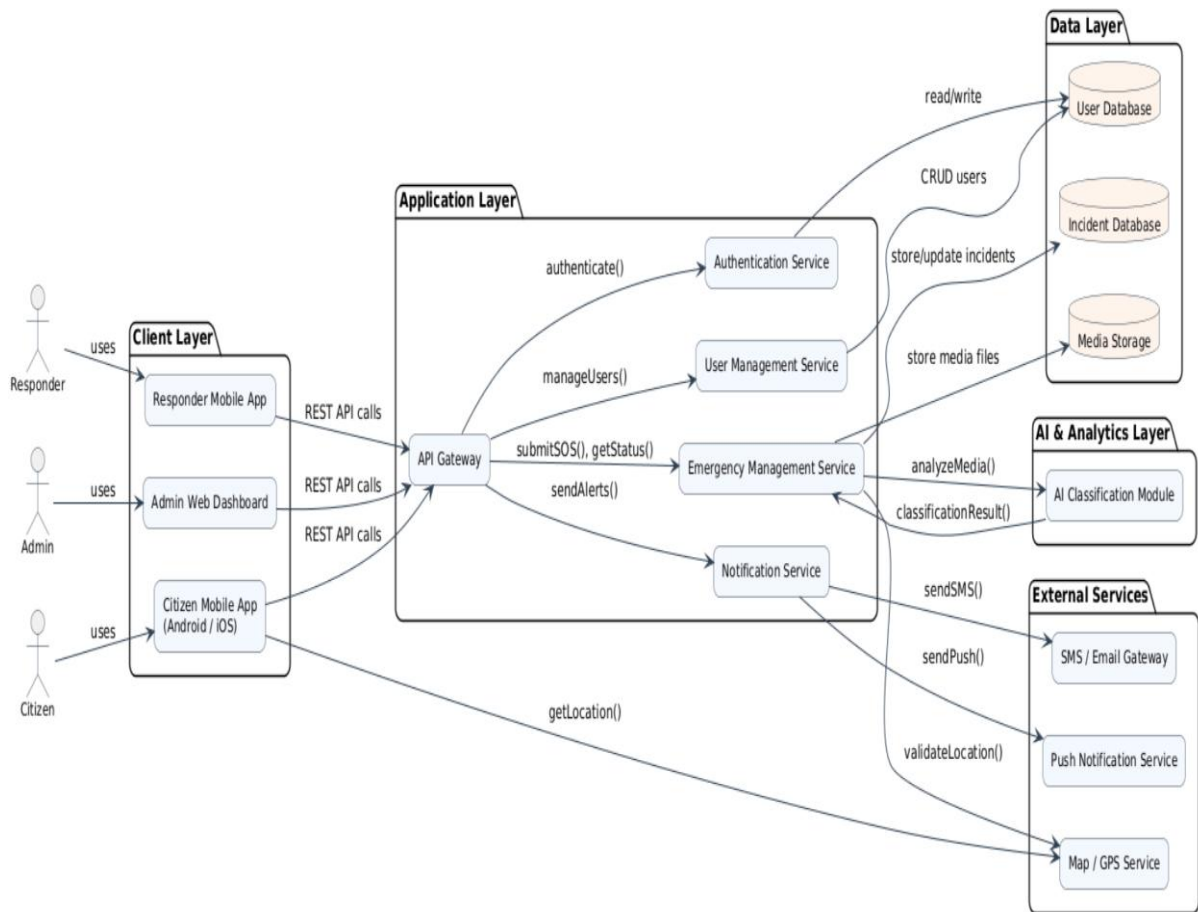


Figure 12 Component Diagram

3.1.3 Deployment Modelling

The deployment model describes how the software components of the system are physically distributed across hardware and networked environments. It defines where each part of the system runs, how components communicate, and how the system ensures reliability, scalability, and security. A well-designed deployment model is critical for supporting real-time emergency reporting, continuous availability, and efficient data processing.

Deployment Environment

1. Client Devices

Client devices represent the user-facing layer of the system and include smartphones (mobile application) and web browsers (web-based interface). These devices are used by citizens, police officers, and administrators to interact with the system.

On smartphones, the system enables rapid emergency reporting, GPS-based location capture, media uploads, and real-time notifications. Web browsers provide a larger interface suitable for police officers and administrators to manage cases, analyze reports, and monitor system activity.

Client devices communicate with the backend servers over secure internet connections using standard web protocols (such as HTTPS). By supporting both mobile and web platforms, the system ensures accessibility, usability, and broad adoption across different user groups.

2. Application Server (Node.js Backend)

The application server hosts the core business logic of the system and acts as the central coordinator between clients, databases, and external services. Implemented using **Node.js**, the backend efficiently handles multiple concurrent requests, making it suitable for real-time and high-traffic environments such as emergency reporting systems.

Key responsibilities of the application server include:

- ❖ Processing crime and emergency reports
- ❖ Enforcing authentication and role-based access control
- ❖ Managing case workflows and status updates
- ❖ Sending notifications to users
- ❖ Acting as an interface to AI and GIS cloud services

The application server exposes secure RESTful APIs that allow client devices to exchange data reliably and in real time. Its centralized role ensures consistency in system behavior and simplifies maintenance and future upgrades.

3. Database Server (MySQL and MongoDB)

The system employs a hybrid database architecture to efficiently manage different types of data.

- ❖ **MySQL** is used for structured, relational data such as user accounts, roles, case records, and system logs. Its relational nature ensures data consistency, integrity, and support for transactional operations.

- ❖ **MongoDB** is used for unstructured or semi-structured data, including multimedia metadata, incident descriptions, and AI-generated analytical results. Its flexible schema allows the system to store varied and evolving data formats efficiently.

By separating structured and unstructured data storage, the deployment improves performance, scalability, and data organization. Database servers are secured with access controls, regular backups, and replication mechanisms to ensure availability and data protection.

4. Cloud Services (AI Processing and GIS Services)

Cloud services extend the system's capabilities beyond local infrastructure by providing scalable and specialized processing resources.

- ❖ **AI processing services** analyse collected crime data to identify trends, predict hotspots, and support decision-making for law enforcement. Cloud-based AI enables high computational performance without overloading the application server.
- ❖ **GIS services** support geographic visualization, location tracking, and map-based analysis of crime incidents. These services enable real-time mapping of reports and improve situational awareness for authorities.

Using cloud services allows the system to scale dynamically based on demand, ensuring reliable performance during peak usage such as emergencies or high reporting periods.

This deployment ensures **high availability**, **data integrity**, and **secure communication** between clients and servers.

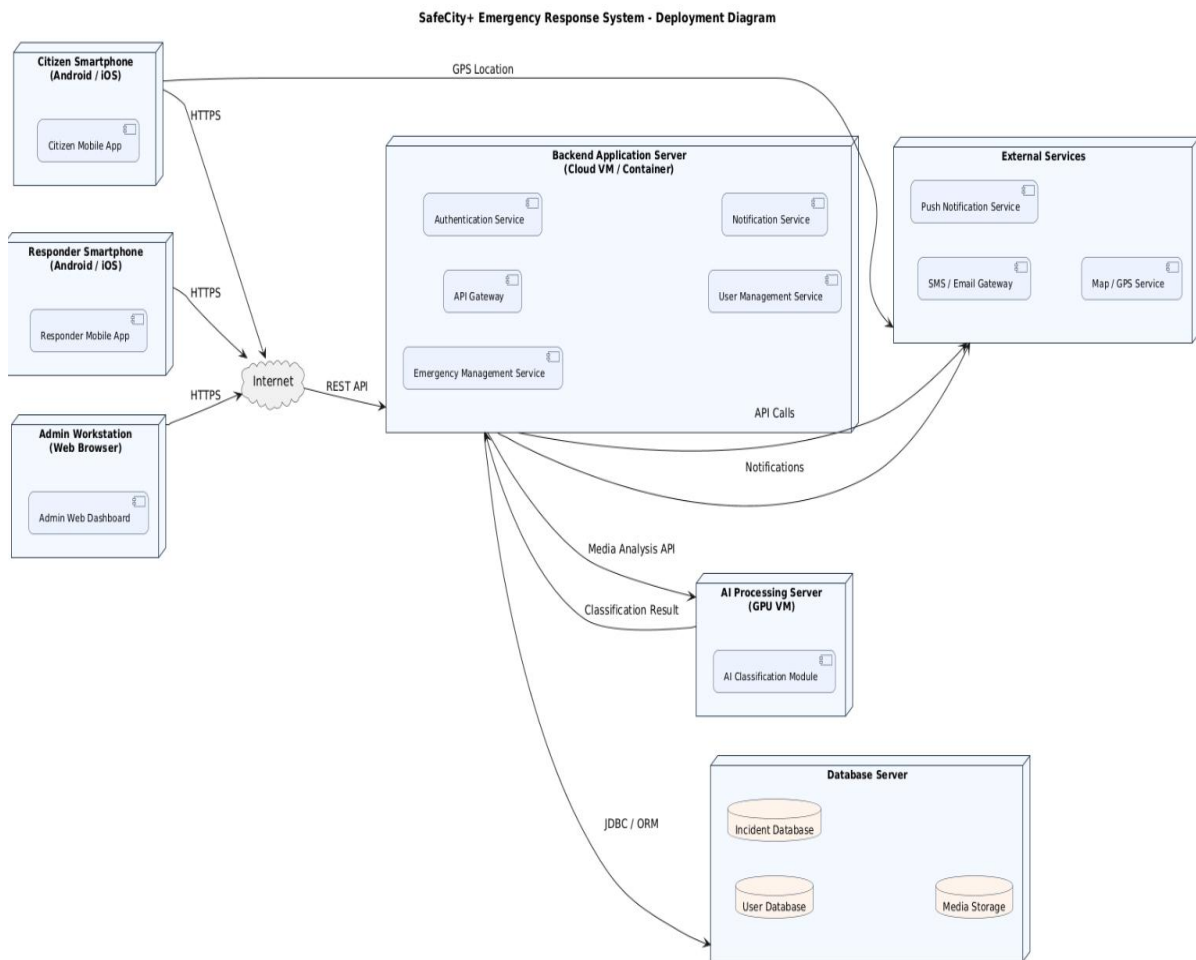


Figure 13 Deployment Diagram

3.2 Detailed Design

3.2.1 Design Class Model

The design class model defines the static structure of the SafeCity+ system by identifying its core classes, their attributes, and the relationships among them. It provides a detailed representation of how data and responsibilities are organized within the system and serves as a foundation for implementation and maintenance. The model follows object-oriented design principles, including encapsulation, inheritance, and association, to promote modularity, reusability, and scalability.

Each class in the system is described with:

- ❖ **Attributes:** Data members, their data types, and access modifiers (public, private, protected).
- ❖ **Methods:** Functions associated with the class, specifying return types and parameters with data types.
- ❖ **Relationships:** Associations between classes (one-to-one, one-to-many, etc.), including inheritance and dependencies.
- ❖ **Multiplicity:** Defines how many instances of one class relate to another (e.g., 1, 0..*, 1..*).

Major Classes

A. User Class

The **User** class represents a generic system user and serves as the superclass for all specific user types. It encapsulates common attributes and behaviors shared across different roles in the system.

Attributes:

- ❖ `userID`: A unique identifier for each user
- ❖ `fullName`: The user's full name
- ❖ `email`: The registered email address used for authentication
- ❖ `password`: Encrypted user password
- ❖ `role`: Defines the user's access level within the system

This class supports authentication and authorization processes and ensures consistent handling of user identity across the system.

B. Citizen Class

The **Citizen** class is a subclass of the User class and represents individuals who use the system to report crimes or emergencies.

Responsibilities:

- ❖ Submit crime and emergency reports
- ❖ Attach media evidence
- ❖ View notification updates related to submitted reports

By inheriting from the User class, the Citizen class reuses authentication and identification attributes while extending system behavior specific to reporting incidents.

C. PoliceOfficer Class

The PoliceOfficer class extends the User class and represents authorized law enforcement personnel.

Responsibilities:

- ❖ Review submitted crime reports
- ❖ Update case status (e.g., Pending, In Progress, Resolved)
- ❖ Manage assigned cases and investigations

This class enables controlled access to operational features required for law enforcement activities while maintaining accountability through user identification.

D. Admin Class

The Admin class also inherits from the User class and represents system administrators responsible for overall system control.

Responsibilities:

- ❖ Manage user accounts and roles
- ❖ Configure system settings
- ❖ Monitor system activity and performance

The Admin class plays a supervisory role, ensuring proper system operation, security enforcement, and policy compliance.

E. CrimeReport Class

The **CrimeReport** class represents an incident or emergency report submitted to the system. It is a central class in the system and connects users, media evidence, and case management processes.

Attributes:

- ❖ `reportID`: Unique identifier for the report
- ❖ `description`: Textual explanation of the incident
- ❖ `date`: Date and time of report submission
- ❖ `status`: Current processing state of the report
- ❖ `location`: Geographic coordinates of the incident

This class supports tracking, analysis, and resolution of reported cases.

F. MediaFile Class

The **MediaFile** class represents multimedia evidence associated with a crime report.

Attributes:

- ❖ `fileID`: Unique identifier for the media file
- ❖ `fileType`: Type of media (image, video, audio)
- ❖ `filePath`: Storage location of the media file

MediaFile objects enhance report credibility by providing visual or audio evidence and are linked directly to CrimeReport instances.

G. Notification Class

The **Notification** class models system-generated messages sent to users.

Attributes:

- ❖ `notificationID`: Unique notification identifier
- ❖ `message`: Notification content
- ❖ `timestamp`: Date and time of notification

Notifications are triggered by system events such as report submission, case assignment, or status updates.

The relationships among classes define how system entities interact and exchange data:

- ❖ A **User** can submit **multiple CrimeReport** instances, establishing a one-to-many relationship.

- ❖ A **CrimeReport** may include **multiple MediaFile** objects, representing a one-to-many association.
- ❖ The **Admin** class maintains management relationships with User accounts and system configurations.
- ❖ The **Notification** class is associated with the User class, allowing multiple notifications to be delivered to a single user.

Inheritance relationships exist between the User class and its subclasses (Citizen, PoliceOfficer, Admin), enabling role specialization while maintaining a common user structure.

SafeCity+ Emergency Response System - Class Diagram

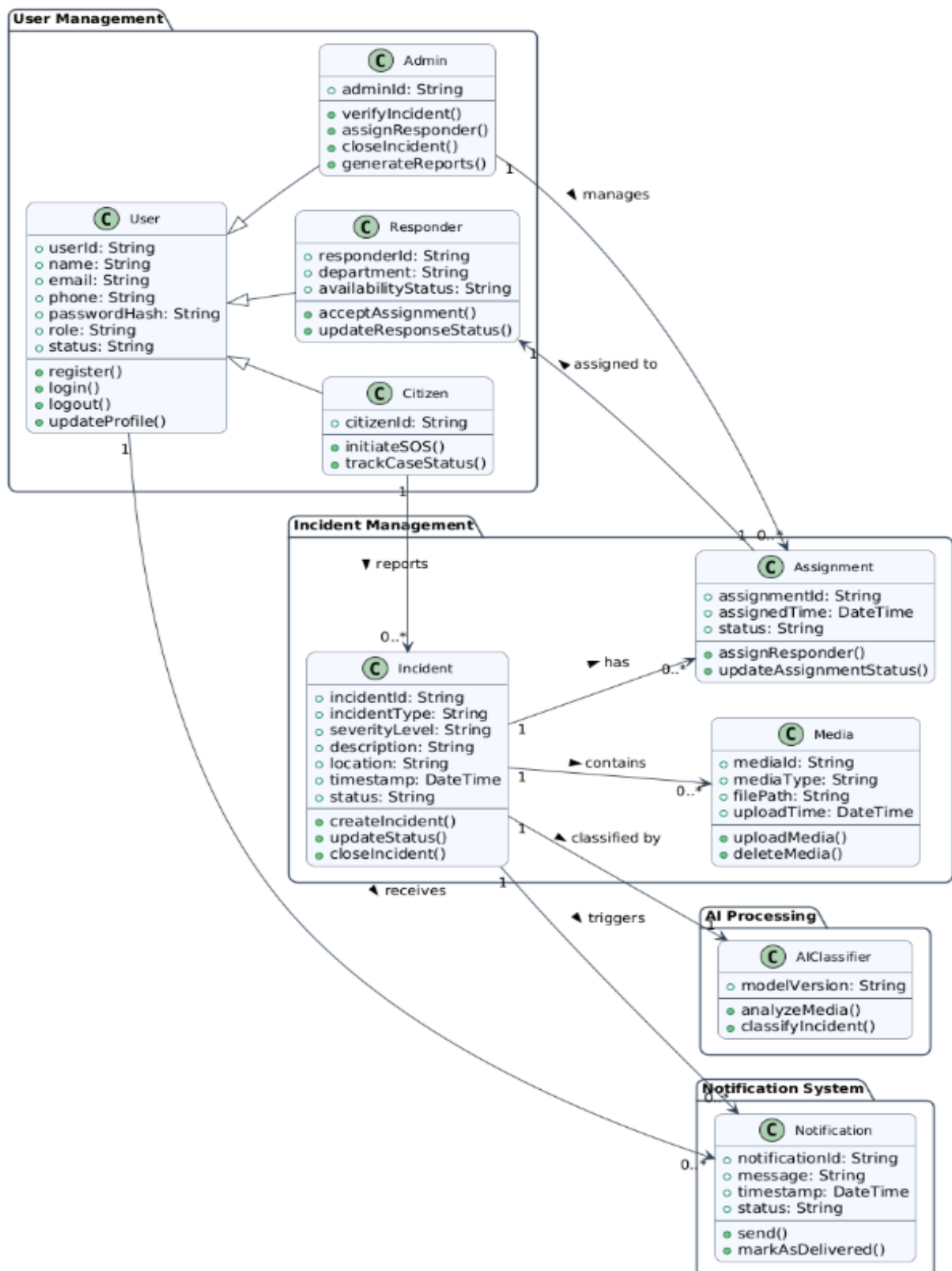


Figure 14: Class Diagram

3.2.2 Persistent Model

The persistent model represents how data is stored in the database.

Main Database Tables / Collections

- ❖ Users
- ❖ CrimeReports
- ❖ MediaFiles
- ❖ Notifications
- ❖ Roles

The system uses:

- ❖ **MySQL** for structured relational data
- ❖ **MongoDB** for unstructured multimedia data

This hybrid approach ensures flexibility and performance.

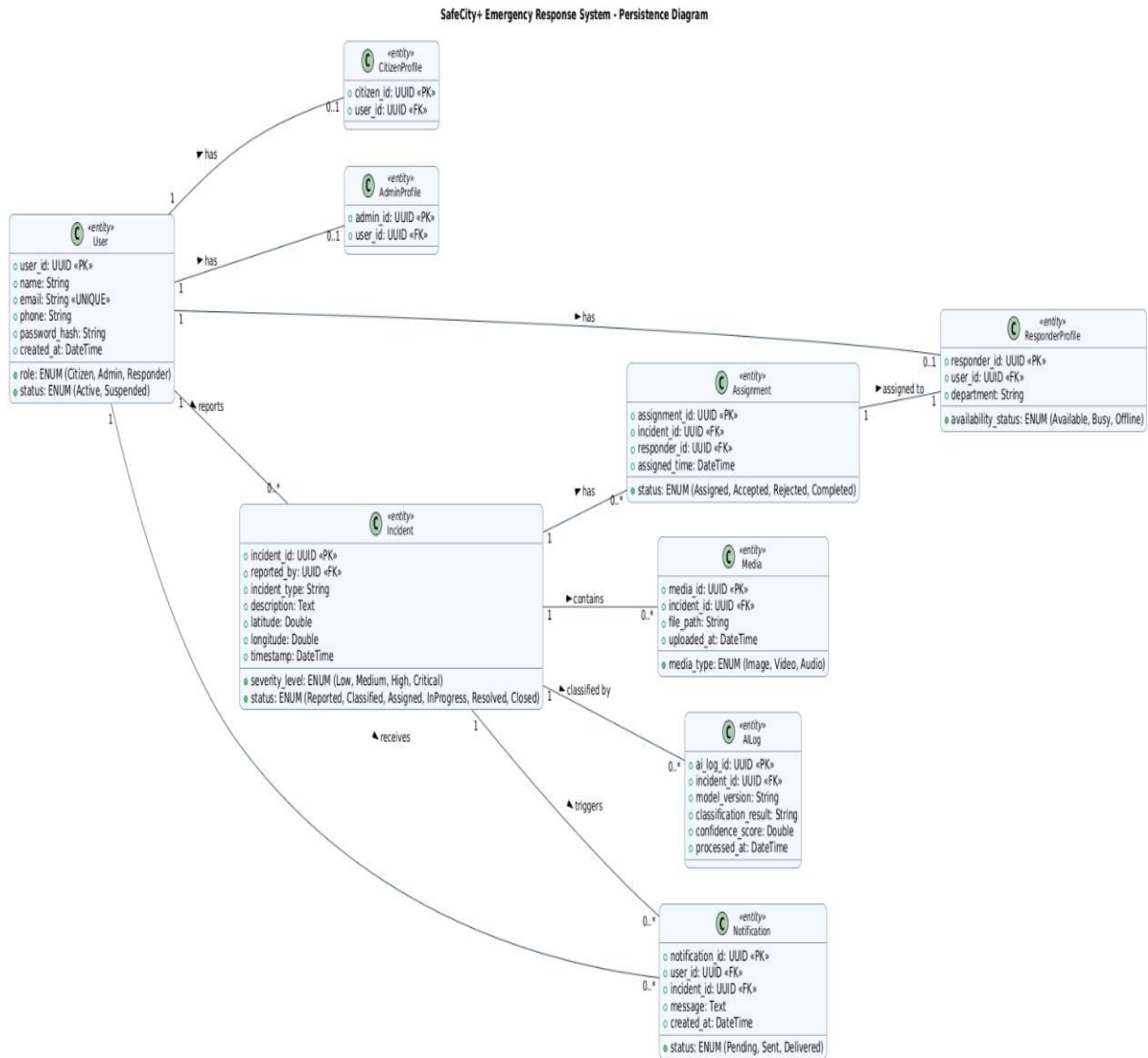


Figure 15 Persistent Diagram (ERD)

3.3 User Interface Design

The user interface of the SafeCity+ system is designed to be simple, intuitive, and user-friendly in order to support efficient interaction under both normal and emergency conditions. Given the diverse range of users, including citizens, police officers, and administrators, the interface emphasizes clarity, accessibility, and ease of navigation. The design prioritizes rapid task completion, particularly for emergency reporting, while maintaining consistency across platforms.

3.3.1 Citizen Mobile Application

The Citizen Mobile Application serves as the primary interaction point for the general public. It is optimized for mobile devices to support on-the-go usage and quick access during emergency situations. The application provides a registration and login screen that enables users to securely create accounts or access existing ones. For urgent situations, the interface also supports direct access to emergency reporting with minimal interaction steps. The crime reporting form is designed with a structured layout that guides users through the submission process. It allows users to enter incident details, attach multimedia evidence, and confirm submission with minimal cognitive load. A map-based location selection feature is integrated to allow users to identify the incident location accurately using GPS or manual map selection. This visual approach reduces errors and improves location accuracy. The case status tracking interface enables citizens to view real-time updates on submitted reports, including status changes and notifications. This transparency improves user trust and engagement with the system.

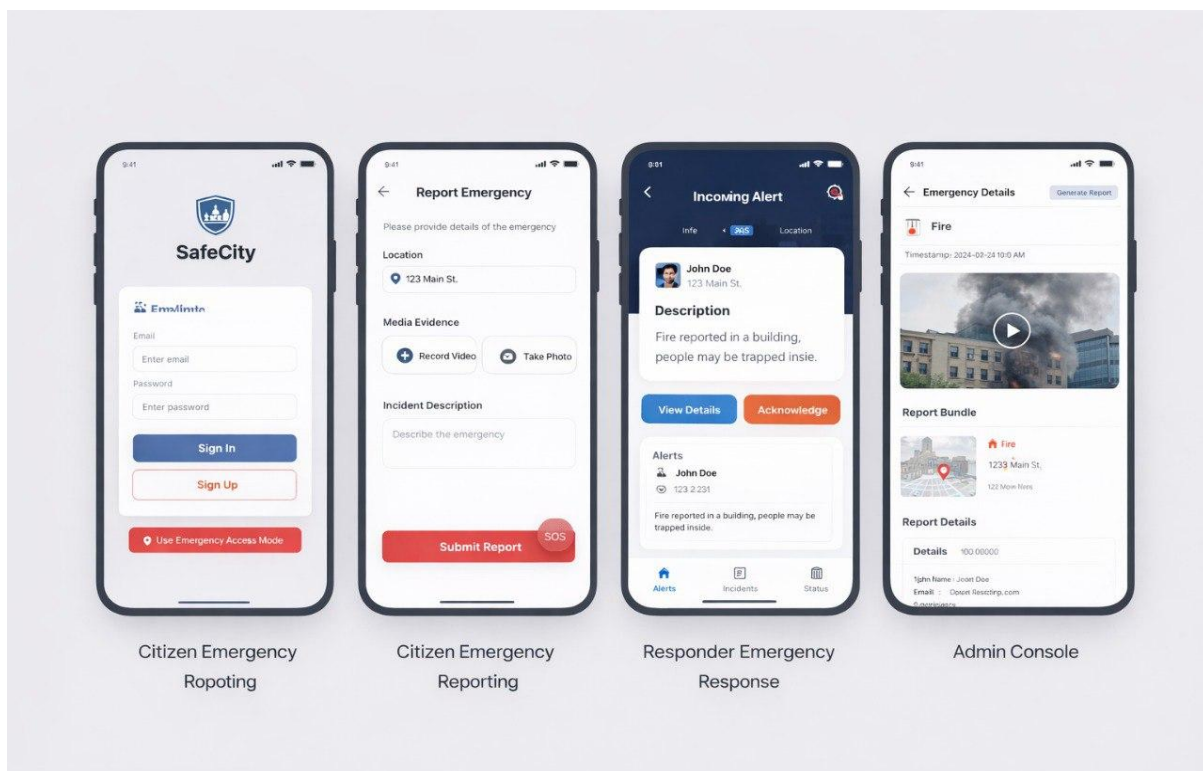


Figure 16 App based system design

3.3.2 Admin & Responder Web Dashboard

The Admin and Police Web Dashboard is designed for operational efficiency and system oversight. It is accessible through modern web browsers and optimized for desktop environments. The dashboard includes a secure login interface that ensures only authorized personnel can access administrative and law enforcement functions. Once authenticated, users are presented with role-specific dashboards. The crime report list and filtering feature allows officers to view, search, and filter reports based on criteria such as date, location, type, or status. This functionality supports rapid case review and prioritization. Case assignment and management tools enable administrators and police supervisors to assign reports to officers, update case statuses, and monitor progress. These features support structured workflows and accountability. An analytics and visualization dashboard presents summarized crime data using charts, graphs, and maps. This visual representation supports decision-making, trend analysis, and resource planning.

The SafeCity+ user interface is guided by established UI/UX design principles to ensure usability and inclusiveness:

- ❖ **Consistency:** Uniform layouts, color schemes, and interaction patterns are maintained across all interfaces to reduce learning time and prevent user confusion.
- ❖ **Minimalism:** Interfaces display only essential elements, reducing visual clutter and enabling users to focus on critical tasks, especially during emergencies.
- ❖ **Accessibility:** The design considers users with varying abilities by supporting readable fonts, sufficient color contrast, and intuitive navigation.
- ❖ **Responsive Design:** The interface adapts to different screen sizes and devices, ensuring seamless interaction across smartphones, tablets, and desktops.

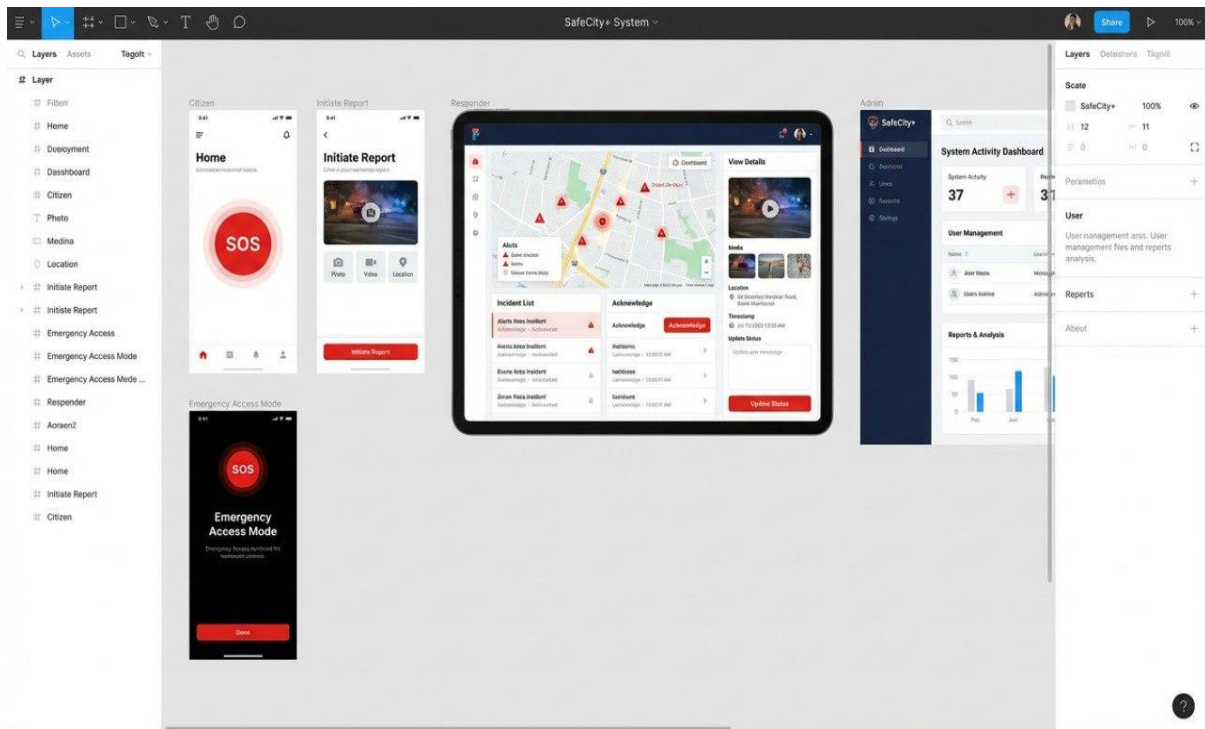


Figure 17 Web based system design

3.4 Access Control and Security Design

Security is a critical component of the SafeCity+ system due to the sensitive nature of crime reports, personal user data, and law enforcement operations. The system is designed with multiple security layers to ensure confidentiality, integrity, and availability of data. These measures protect the system against unauthorized access, data breaches, and malicious attacks while maintaining reliable and secure communication between system components.

Authentication

Authentication is the process of verifying the identity of users before granting access to the system. SafeCity+ implements secure authentication mechanisms to ensure that only legitimate users can access protected system resources.

User login credentials are protected using encrypted password storage, ensuring that plaintext passwords are never stored in the database. During login, credentials are validated securely against stored encrypted values.

The system employs JSON Web Token (JWT)-based authentication to manage user sessions. After successful login, the server issues a signed token that is used to authenticate subsequent requests. This approach enables stateless session management, improves scalability, and reduces the risk of session hijacking. Tokens are validated on each request to ensure authenticity and integrity.

Authorization

Authorization determines what actions an authenticated user is permitted to perform within the system. SafeCity+ uses Role-Based Access Control (RBAC) to enforce authorization policies consistently and securely.

Each user is assigned a predefined role that determines accessible system functions:

- ❖ **Citizens** are authorized to submit crime or emergency reports and view updates related to their submissions.
- ❖ **Responder** are authorized to review reports, manage case assignments, and update case statuses.
- ❖ **Administrators** have full system control, including user management, system configuration, and monitoring activities.

RBAC ensures separation of responsibilities and prevents unauthorized access to sensitive operations, thereby reducing security risks and improving accountability.

Data Security

Data security focuses on protecting information during transmission and storage. All data exchanged between client devices and backend servers is secured using encrypted communication protocols (HTTPS), preventing interception and tampering during transmission.

Database access is restricted through authentication credentials and access control policies, ensuring that only authorized system components can interact with stored data. Sensitive data is protected against unauthorized exposure through controlled access mechanisms.

Additionally, the system implements input validation and structured error handling to prevent malformed or malicious data from entering the system. This reduces the risk of application-level attacks and improves system stability.

System Security Measures

Security is a critical component of the SafeCity+ system due to the sensitive nature of crime reports, personal user data, and law enforcement operations. The system is designed with multiple security layers to ensure confidentiality, integrity, and availability of data. These measures protect the system against unauthorized access, data breaches, and malicious attacks while maintaining reliable and secure communication between system components.

References

- ❖ Caragliu, A., Del Bo, C., & Nijkamp, P. (2011). Smart cities in Europe. *Journal of Urban Technology*, 18(2), 65-82.
- ❖ Nam, T., & Pardo, T. A. (2011). Conceptualizing smart city with dimensions of technology, people, and institutions. *Proceedings of the 12th Annual International Digital Government Research Conference: Digital Government Innovation in Challenging Times*, 282-291.
- ❖ Harrison, C., Eckman, B., Hamilton, R., Hartswick, P., Kalagnanam, J., Paraszczak, J., & Williams, P. (2010). Foundations for Smarter Cities. *IBM Journal of Research and Development*, 54(4), 1-16.
- ❖ Meier, P. (2015). *Digital Humanitarians: How Big Data Is Changing the Face of Humanitarian Response*.* CRC Press.
- ❖ Bahir Dar City Administration. (2024). *Smart Bahir Dar Initiative: Integrated Urban Development Plan*.